INSPECTION GÉNÉRALE / 30 OCTOBRE 2026

AeroMind — Audit technique

Inspection mécanique & matérielle approfondie en vue d'une opérationnalisation à 100 %.

Rapport d'audit indépendant couvrant le backend Python FastAPI (41 modules), le frontend React/Vite (≈ 43 000 lignes), la suite de tests (30 fichiers pytest + 57 fichiers vitest), l'historique des audits (75+ rapports) et la chaîne DevOps (Docker, Render, CI GitHub Actions, migrations SQL).

12	14	18	9
CRITIQUES	HAUTS	MOYENS	BAS

PROJET	AeroMind — LabBench 01 (Option C FAST-TRACK)
PÉRIMÈTRE	Backend · Frontend · Tests · DevOps · Sécurité
CIBLE	Production Render + PostgreSQL
PRÉPARÉ PAR	Z.ai · Inspection statique read-only

Table des matières

Sommaire exécutif et verdict 5

Verdict global . . . 5
Répartition des défauts par axe et sévérité . . . 5
Les cinq défauts bloquants à corriger en priorité absolue . . . 6
Méthodologie d'inspection en bref . . . 7

Contexte et méthodologie d'inspection 8

Présentation du projet AeroMind . . . 8
Périmètre d'inspection . . . 8
Limites de l'inspection . . . 9
Critères de sévérité . . . 9

Matrice de synthèse des défauts 11

Tous les défauts CRITICAL . . . 11
Tous les défauts HIGH . . . 12
Top 5 bloquants production — synthèse visuelle . . . 13

Inspection Backend Python — Findings détaillés 15

4.1 Architecture & structure des modules . . . 15
4.2 Couche de données — divergences PostgreSQL / TiDB . . . 16
4.3 Authentification & autorisation . . . 17
4.4 Posture sécurité — endpoints matériel & dashboard . . . 19
4.5 Intégration MAVLink — pool, telemetry consent, mission upload . . . 20
4.6 Modes opérationnels & gates G0-G5 . . . 21
4.7 Gestion d'erreurs & logging . . . 22
4.8 Configuration & variables d'environnement . . . 22
4.9 Qualité code & dette technique . . . 22
4.10 Dépendances Python . . . 23
4.11 Readiness déploiement — Dockerfile & render.yaml . . . 23

Inspection Frontend React — Findings détaillés 25

5.1 Architecture & structure du projet . . . 25
5.2 Configuration de build . . . 26
5.3 Routing & pages . . . 26

5.4 Inventaire des composants . . . 27
5.5 Gestion d'état . . . 27
5.6 Flux d'authentification . . . 28
5.7 Internationalisation — deux systèmes parallèles . . . 29
5.8 Client API & tRPC . . . 30
5.9 UI MAVLink & télémétrie . . . 30
5.10 Jumeau mécanique / CAD . . . 31
5.11 Module LabBench 01 . . . 31
5.12 Sécurité client-side . . . 31
5.13 Accessibilité & UX . . . 32
5.14 Qualité de code . . . 33
5.15 Tests frontend — couverture critique . . . 33
5.16 Build readiness . . . 33

Inspection Tests & Historique d'audit 35

6.1 Inventaire des tests . . . 35
6.2 Couverture backend — modules sans test dédié . . . 35
6.3 Couverture frontend — composants sans test . . . 36
6.4 Qualité des tests — flakiness, skips, hardcoded . . . 36
6.5 Intégration CI — .github/workflows/ci.yml . . . 36
6.6 Chronologie des audits et livraisons . . . 37
6.7 Défauts récurrents — identifiés à plusieurs reprises sans correction . . . 38
6.8 Statut réel SEC-01 à SEC-08 — README vs audit vs code . . . 38
6.9 Contradictions entre audits successifs . . . 39
6.10 Lots A-F — statut réel . . . 40

Inspection DevOps & Déploiement 41

7.1 Dockerfile — multi-stage, mais améliorations nécessaires . . . 41
7.2 docker-compose.yml — secret hardcoded, pas de networks . . . 42
7.3 render.yaml — Blueprint Render . . . 42
7.4 .env.example — variables critiques manquantes . . . 44
7.5 CI/CD workflows — scanners sécurité silenciés . . . 44
7.6 Migrations de base de données . . . 45
7.7 Backup & disaster recovery . . . 47
7.8 Concerns opérationnels — observabilité, rate limiting, CSP . . . 47

Plan de remédiation P0 — Bloquants production 49

P0-01 — Définir current_user_or_401 (B-CRIT-01) . . . 49
P0-02 — Fixer _hmac_secret (B-CRIT-02) . . . 49
P0-03 — Passer owner_id et project_id (B-CRIT-04, B-CRIT-05, B-HIGH-01) . . . 50
P0-04 — Enforcer CSRF sur routes state-changeantes (B-CRIT-03) . . . 50
P0-05 — Scoper dashboard par Project.user_id (B-HIGH-02) . . . 51
P0-06 — Synchroniser migration TiDB (B-CRIT-06, D-CRIT-10) . . . 51
P0-07 — Persister telemetry consent en DB (B-HIGH-04) . . . 52
P0-08 — Vérifier gates G0-G5 (B-HIGH-06) . . . 53
P0-09 — AUTO_CREATE_SCHEMA=false + preDeployCommand (B-HIGH-09) . . . 54
P0-10 — Supprimer mcgregoralvin14@gmail.com (D-CRIT-08, B-SEC-05) . . . 54
P0-11 — CI scanners bloquants (D-CRIT-09) . . . 55
P0-12 — Step-up roles server-side (T-HIGH-27, SEC-08) . . . 56
Synthèse sprint 1 . . . 57

Plan de remédiation P1 — Haute priorité 58

P1-01 — Adopter Alembic pour les migrations . . . 58
P1-02 — Pin Actions à SHA et permissions least-privilege . . . 58
P1-03 — Sentry + structlog + rate limiting . . . 58
P1-04 — Content-Security-Policy header . . . 58
P1-05 — Remplacer tRPC shim par fetchTRPC typé . . . 59
P1-06 — Lazy-load Home + extraction sous-vues . . . 59
P1-07 — Suite Playwright E2E (6 scénarios) . . . 59
P1-08 — Ajouter pgserver à requirements-dev.txt . . . 59
P1-09 — Coverage threshold Vitest + raise maxfail . . . 59
P1-10 — Cleanup __manus__ leftovers + broken Tailwind class . . . 60
P1-11 — Document secret rotation runbook . . . 60
P1-12 — Écrire docs/PROCEDURE-INCIDENT-FR.md . . . 60
P1-13 — Mettre à jour le manuel opérateur . . . 60
Synthèse sprints 2-3 . . . 60

Plan de remédiation P2/P3 — Medium et Low 61

P2 — Défauts MEDIUM (priorité parallèle) . . . 61
P3 — Défauts LOW (à traiter quand l'occasion se présente) . . . 63

Roadmap vers 100% opérationnalisation 64

Vue d’ ensemble — 6 sprints 64
Sprint 1 — P0 Critical Fixes (semaines 1-2) 64
Sprint 2 — P1 DevOps Hardening (semaines 3-4) 64
Sprint 3 — P1 Frontend Refactor (semaines 5-6) 65
Sprint 4 — P1 Observability (semaines 7-8) 65
Sprint 5 — P1 Tests E2E (semaines 9-10) 65
Sprint 6 — Pre-production Deploy (semaines 11-12) 65
KPIs globaux de succès 66

Conclusion et recommandations finales **67**

Bilan global 67
Cinq actions top-priority résumées 67
Recommandations finales 68
Mot de la fin 69

CHAPITRE 1 · SYNTHÈSE

Sommaire exécutif et verdict

Le présent rapport consigne le résultat d'une inspection générale, complète et approfondie du projet **AeroMind** — cockpit d'ingénierie drone livré sous forme d'archive AeroMind_modernisation_auth_https_LabBench_2026-10-29.tar.gz. L'inspection a porté sur quatre axes : le backend Python (FastAPI), le frontend React/Vite, la suite de tests et l'historique des audits, ainsi que la chaîne DevOps (Docker, Render, CI GitHub Actions, migrations SQL). La méthodologie retenue est exclusivement statique et read-only : aucun fichier n'a été modifié, aucune exécution de code n'a été lancée.

Verdict global

Le projet n'est pas, en l'état, opérationnalisable à 100 % en production. L'architecture générale est saine — séparation claire des modules backend, multi-stage Docker avec utilisateur non-root, RBAC à six rôles, gates G0-G5 définies, migrations PostgreSQL/TiDB documentées — mais l'inspection révèle **53 défauts actionnables** dont **12 critiques** qui bloquent ou compromettent gravement la mise en production. Plusieurs de ces défauts ont déjà été identifiés dans des audits internes antérieurs (du 13 octobre au 28 octobre 2026) sans être réellement corrigés — le rapport de livraison du 26 octobre a même ré-introduit explicitement l'adresse Gmail personnelle mcgregoralvin14@gmail.com comme super-admin par défaut, contredisant la note du README qui prétendait l'avoir supprimée.

Les chapitres suivants détaillent, axe par axe, les défauts identifiés avec leur sévérité, leur emplacement précis (fichier, ligne), leur impact concret et le correctif recommandé. Le chapitre 10 (« Plan P0 ») consolide les douze corrections bloquantes à appliquer **avant tout déploiement production**. Le chapitre 13 propose une roadmap séquencée en six sprints de deux semaines pour atteindre une mise en production défendable sur Render + PostgreSQL, conformément au périmètre opérationnel cible.

Répartition des défauts par axe et sévérité

Le tableau ci-dessous synthétise la répartition des défauts par axe d'inspection et par niveau de sévérité. Les chiffres incluent les défauts déjà identifiés dans les audits internes antérieurs mais toujours non corrigés (statut NOT FIXED ou PARTIAL), ainsi que les nouveaux défauts détectés au cours de la présente inspection. Les défauts de niveau INFO (bonnes pratiques relevées) ne sont pas comptabilisés dans ce total.

Axe d'inspection	CRITICAL	HIGH	MEDIUM	LOW	Total
Backend Python	4	5	6	2	17

Axe d'inspection	CRITICAL	HIGH	MEDIUM	LOW	Total
Frontend React	3	7	8	6	24
Tests & audits	1	5	4	3	13
DevOps & déploiement	4	4	6	4	18
Total défauts	12	21	24	15	72

Tableau 1 — Répartition des défauts par axe d'inspection et sévérité.

Les cinq défauts bloquants à corriger en priorité absolue

Les cinq défauts ci-dessous sont les plus urgents : chacun bloque, à lui seul, une fonctionnalité critique ou introduit une vulnérabilité de sécurité inacceptable en production. Ils doivent être corrigés **avant tout autre travail fonctionnel** et constituent le contenu du sprint 1 du plan de remédiation (chapitre 10).

CRITIQUE 1 — Backend LABBENCH totalement injoignable

L'endpoint `POST /api/lab/sessions/step-up`, unique porte d'entrée du routeur canonique `/api/lab/*` (12 endpoints, 737 lignes), importe la fonction `current_user_or_401` qui n'existe pas dans `web/app_auth_helpers.py`. Conséquence : **ImportError à chaque appel**. Aucune session step-up ne peut être émise, aucune DroneRevision créée, aucun TestRun démarré, aucune gate approuvée.

CRITIQUE 2 — Hardware tokens signés avec une clé constante

La fonction `_hmac_secret()` (`web/labbench_auth.py:381-383`) utilise `getattr(settings, "AEROMIND_SESSION_SECRET", "")` mais Pydantic Settings expose l'attribut Python `session_secret`, pas l'alias `AEROMIND_SESSION_SECRET`. Résultat : la fonction retourne **toujours** `"insecure-dev-only"`. Tous les tokens matériels du monde sont signés avec la même clé publique. Attaque : forge de token HMAC à partir de n'importe quel dump de la table `hardware_tokens`.

CRITIQUE 3 — Endpoints /api/hardware/pool/register et /api/hardware/geofence injoignables

L'appel `pool.register(drone_id, drone_name, connection_url)` (`web/app.py:1202`) omet les arguments obligatoires `owner_id` et `project_id` : le pool lève systématiquement `HardwareCommandError(400)`. Idem pour `set_geofence(geofence)` (`web/app.py:1427`) qui omet `owner_id` → TypeError à chaque appel. Aucun drone ne peut être enregistré dans le pool, aucune géofence ne peut être posée, et comme la géofence est fail-closed, **aucune commande MAVLink ne peut jamais être autorisée**.

CRITIQUE 4 — Adresse Gmail personnelle en super-admin par défaut en production

Trois fichiers référencent encore `mcgregoralvin14@gmail.com` comme valeur par défaut de `AEROMIND_SUPERADMIN_EMAILS`, `AEROMIND_TRIAL_EMAILS` et `DEFAULT_DEMO_EMAIL` : `web/config.py:55,59`, `web/admin.py:65`, `.env.example:23-25` et `render.yaml:49-54`. Tout déploiement Render qui n'écrase pas ces variables accorde les droits super-admin à cette adresse. Le rapport `LIVRAISON-CORRECTION-IDENTITE-AEROMIND-LABBENCH.md` du 26 octobre a explicitement ré-introduit cette adresse, contredisant le README § « Premier administrateur ».

CRITIQUE 5 — Scanners de sécurité CI tous silenciés par `|| true`

Le workflow `.github/workflows/ci.yml` exécute bien `pip-audit`, `pnpm audit` et `Trivy`, mais tous trois sont suffixés par `|| true` (lignes 42, 82, 108) ou `exit-code: "0"`. Conséquence : **une CVE critique RCE ne bloquerait pas la fusion d'une PR**. Le paramètre `--ignore-vuln GHSA-XXXX` est un placeholder non remplacé. Aucune branche protégée n'exige que les 5 jobs CI passent.

Méthodologie d'inspection en bref

L'inspection a été conduite par lecture statique du code source extrait de l'archive (49 MB, 14 répertoires racines, environ 43 000 lignes TypeScript/TSX côté frontend et 41 modules Python côté backend). Quatre agents d'inspection indépendants ont été lancés en parallèle, chacun spécialisé sur un axe : backend, frontend, tests/audits, DevOps. Chaque agent a produit un rapport structuré avec, pour chaque défaut : titre court, sévérité, fichier(s) source et ligne(s), impact concret et correctif recommandé. Les résultats ont ensuite été consolidés, dédupliqués et croisés avec les 75 rapports d'audit internes déjà présents dans l'archive (du 13 octobre au 29 octobre 2026) pour détecter les défauts récurrents et les contradictions entre audits successifs.

Les défauts ont été classés selon cinq niveaux de sévérité. **CRITICAL** désigne les défauts qui bloquent une fonctionnalité essentielle ou introduisent une vulnérabilité exploitable ; ils doivent être corrigés avant tout déploiement. **HIGH** correspond à des défauts qui compromettent sérieusement la qualité de production sans bloquer immédiatement ; correction dans le sprint suivant. **MEDIUM** et **LOW** concernent des défauts de qualité, de maintenabilité ou de robustesse à traiter en parallèle. **INFO** signale des bonnes pratiques effectivement respectées (non comptabilisées dans le total).

CHAPITRE 2 · MÉTHODOLOGIE

Contexte et méthodologie d'inspection

Présentation du projet AeroMind

AeroMind est un cockpit d'ingénierie drone destiné à la conception, la simulation et l'essai de drones. L'application se présente sous la forme d'une plateforme web fullstack composée d'un backend Python FastAPI (41 modules, 1 666 lignes pour le seul fichier `web/app.py`), d'un frontend React 19 / Vite 7 / TypeScript 5.9 avec tRPC client 11.6 et Three.js 0.186 pour le jumeau mécanique, et d'une couche de données PostgreSQL ou TiDB avec migrations SQL datées. Le projet embarque également une intégration MAVLink (via pymavlink 2.4.49) pour la communication avec des autopilotes ArduPilot, un mode SITL (Software In The Loop) et un module LABBENCH 01 (banque d'essai) avec sessions step-up MFA, RBAC à six rôles (reader, designer, analyst, operator, approver, admin), gates de validation G0 à G5, et géofence fail-closed.

L'archive inspectée, datée du 29 octobre 2026, correspond à la version livrée après la dernière vague de modernisation (navigation satellite HTTPS, accès MFA, modernisation de LABBENCH 01). Elle contient également 75 rapports d'audit, de livraison et de modernisation accumulés depuis le 13 octobre 2026, ce qui constitue à la fois une traçabilité exceptionnelle et un signal d'alerte : la persistance de certains défauts à travers plusieurs audits successifs suggère que les cycles de correction n'ont pas tous été menés à terme.

Périmètre d'inspection

L'inspection couvre quatre axes indépendants, chacun confié à un agent dédié :

Axe 1 — Backend Python. Ensemble des modules du répertoire `web/`, migrations SQL, scripts utilitaires, Dockerfile et configuration. L'accent est mis sur l'architecture modulaire, la surface d'API, la couche de données, l'authentification (OIDC Entra + MFA locale + step-up LABBENCH), la posture de sécurité (CSRF, CORS, validation Pydantic, injection SQL, gating MAVLink), l'intégration MAVLink, les modes opérationnels DEMO/SIMULATION/SITL/HIL/HARDWARE/OFFLINE, le traitement d'erreurs, la configuration et la qualité de code.

Axe 2 — Frontend React. Application Vite (`frontend/`) : configuration de build (package.json, tsconfig, vite.config), routage, inventaire des composants, gestion d'état (ConceptConfig, React Query/tRPC), flux d'authentification (OIDC, MFA locale, dev-login, RBAC client), implémentation i18n (deux systèmes parallèles représentant 8 200 clés × 2 langues), client API tRPC, UI MAVLink/télémétrie, jumeau mécanique Three.js, module LabBench 01, sécurité client (XSS, CSP, stockage de tokens), accessibilité, qualité de code et couverture de tests.

Axe 3 — Tests et historique d'audits. Suite pytest backend (30 fichiers, ~210 tests), suite vitest frontend (57 fichiers, ~339 tests), intégration CI (workflow `.github/workflows/ci.yml`), et analyse exhaustive des 75 rapports d'audit accumulés depuis le 13 octobre 2026. L'objectif est de distinguer ce qui a effectivement été corrigé, ce qui reste partiel, et ce qui a été identifié à plusieurs reprises sans l'être — signal d'alerte sur la fiabilité des déclarations de closure.

Axe 4 — DevOps et déploiement. Dockerfile (multi-stage, non-root, healthcheck), docker-compose.yml, render.yaml Blueprint (Render web service + PostgreSQL), .env.example, .dockerignore, CI/CD workflows GitHub Actions, migrations SQL (versionnage, idempotence, rollback), readiness checklists, backup & disaster recovery, concerns opérationnels (log rotation, secret rotation, TLS/CORS, rate limiting, DDoS).

Limites de l'inspection

L'inspection est statique et read-only : aucun fichier n'a été modifié, aucune exécution de code n'a été lancée, aucune base de données réelle n'a été montée. Les conclusions sur le comportement à l'exécution (par exemple, le succès effectif d'un `pnpm run build`) sont déduites de la lecture du code et de la configuration, mais non vérifiées empiriquement. Les chiffres de tests passants cités par les rapports de livraison antérieurs (par exemple « 281 tests / 56 fichiers » pour la livraison du 29 octobre) ne sont pas re-validés ; l'inspection note cependant que la livraison du 29 octobre déclare explicitement ne pas avoir exécuté la suite pytest backend (« pytest n'est pas installé dans l'environnement de livraison »), ce qui signifie que l'état de l'archive n'est pas vérifié côté backend.

L'inspection ne couvre pas : (a) la conformité réglementaire EASA/FAA, qui nécessiterait une revue par un expert du domaine ; (b) la sécurité cryptographique approfondie (audit formel des schémas HMAC/PBKDF2/AES-GCM par un cryptographe) ; (c) la performance sous charge (tests de charge à mener en pré-production) ; (d) l'audit de l'infrastructure cloud Render elle-même (configuration réseau, sauvegardes automatiques, isolation multi-tenant au niveau Render).

Critères de sévérité

Les défauts sont notés selon une échelle à cinq niveaux, calibrée sur le risque d'impact opérationnel en production :

CRITICAL — Défaut qui bloque une fonctionnalité essentielle (endpoint injoignable, import cassé) ou qui introduit une vulnérabilité de sécurité exploitable (forge de token, escalade de privilèges, fuite cross-tenant). Doit être corrigé avant tout déploiement.

HIGH — Défaut qui compromet sérieusement la qualité de production (fuite de données, dégradation silencieuse, absence de test sur un flux critique) sans bloquer immédiatement le service. Correction dans le sprint suivant.

MEDIUM — Défaut de qualité (configuration sous-optimale, dette technique, incohérence de documentation) qui n'impacte pas la sécurité mais alourdit la maintenabilité. Traitement parallèle.

LOW — Défaut mineur (variable mal nommée, commentaire obsolète, dépendance inutilisée). À corriger quand l'occasion se présente.

INFO — Bonne pratique effectivement respectée (multi-stage Docker, lockfile committé, OAuth `__Host-` cookie). Non comptabilisée dans le total des défauts.

CHAPITRE 3 · MATRICE DE SYNTHÈSE

Matrice de synthèse des défauts

Ce chapitre consolide l'ensemble des défauts identifiés par les quatre axes d'inspection en une matrice unique, classée par sévérité décroissante puis par axe. Chaque défaut porte un identifiant stable (par exemple **B-CRIT-01** pour Backend Critical 01, **F-HIGH-03** pour Frontend High 03), référencé dans les chapitres détaillés (4 à 7) et dans le plan de remédiation (chapitres 10 à 12).

Tous les défauts CRITICAL

Les douze défauts CRITICAL sont les seuls qui **bloquent à eux seuls la mise en production**. Ils doivent tous être corrigés dans le sprint 1 (chapitre 10). Aucun déploiement Render ne doit être lancé tant que l'un d'entre eux subsiste.

ID	Titre court	Axe	Fichier(s) source	Statut
B-CRIT-01	current_user_or_401 inexistant — /api/lab/* injoignable	Backend	web/labbench_canonical_api.py:174	NOT FIXED
B-CRIT-02	HMAC token matériel utilise "insecure-dev-only"	Backend	web/labbench_auth.py:381-383	NOT FIXED
B-CRIT-03	CSRF verify_csrf_double_submit jamais appelé + SameSite=None	Backend	web/labbench_auth.py:146 ; web/oidc.py:94	NOT FIXED
B-CRIT-04	/api/hardware/pool/register omet owner_id → 400 systématique	Backend	web/app.py:1202	NOT FIXED
B-CRIT-05	/api/hardware/geofence PUT omet owner_id → TypeError	Backend	web/app.py:1427	NOT FIXED
B-CRIT-06	Migration TiDB divergente (data_json manquant, id VARCHAR(36) trop court)	Backend	migrations/2026-10-25_canonical_labbench_schema_tidb.sql:36,189-192	NOT FIXED
B-CRIT-07	TIDB_SCHEMA.sql couvre 3 tables sur ~30	DevOps	TIDB_SCHEMA.sql	NOT FIXED
D-CRIT-08	mcgregoralvin14@gmail.com en super-admin render.yaml	DevOps	render.yaml:49-54 ; .env.example:23-25	NOT FIXED
D-CRIT-09	Tous les scanners CI silenciés (|| true, exit-code 0)	DevOps	.github/workflows/ci.yml:42,82,108	NOT FIXED
D-CRIT-10	Migration TiDB non idempotente + tables manquantes	DevOps	migrations/2026-10-25_canonical_labbench_schema_tidb.sql	NOT FIXED
F-CRIT-11	tRPC AppRouter = any — contrat API non typé	Frontend	frontend/server/routers.ts:1-2 ; frontend/client/src/lib/trpc.ts:7	NOT FIXED
F-CRIT-12	Session token OIDC mirroré en sessionStorage (XSS exfil)	Frontend	frontend/client/src/lib/sessionToken.ts:1-47 ; main.tsx:52-73	NOT FIXED

Tableau 2 — Liste exhaustive des défauts CRITICAL (12).

Tous les défauts HIGH

Les vingt et un défauts HIGH ne bloquent pas la mise en service mais compromettent sérieusement la robustesse opérationnelle ou la sécurité. Ils doivent être corrigés dans les sprints 2 et 3 du plan de remédiation. Plusieurs d'entre eux (par exemple le monolithe Home.tsx ou l'absence de tests E2E navigateur) sont récurrents depuis le 14 octobre 2026 sans avoir été traités.

ID	Titre court	Axe	Statut
B-HIGH-01	Pool unregister/pilot n'appliquent pas owner_id (cross-tenant)	Backend	NOT FIXED
B-HIGH-02	Dashboard recent_alerts non scopé par Project.user_id	Backend	NOT FIXED
B-HIGH-03	Pas de JIT provisioning — utilisateurs Entra non pré-registrés bloqués	Backend	NOT FIXED
B-HIGH-04	Telemetry consent en mémoire uniquement (multi-worker casse)	Backend	NOT FIXED
B-HIGH-05	assert_no_physical_command / assert_persisted_or_demo jamais appelés	Backend	NOT FIXED
B-HIGH-06	Gates G0-G5 non vérifiées dans le path commande matériel	Backend	NOT FIXED
B-HIGH-07	71 handlers "except Exception" silencieux	Backend	NOT FIXED
B-HIGH-08	tRPC bridge async + SessionLocal sync → event loop bloqué	Backend	NOT FIXED
B-HIGH-09	render.yaml AUTO_CREATE_SCHEMA=true en prod contredit la doc	DevOps	NOT FIXED
B-HIGH-10	AEROMIND_ALLOWED_HOSTS wildcard accepte sous-domaines attaquant	DevOps	NOT FIXED
B-HIGH-11	.env.example incomplet (8 vars critiques manquantes)	DevOps	NOT FIXED
D-HIGH-12	POSTGRES_PASSWORD hardcoded dans docker-compose.yml	DevOps	NOT FIXED
D-HIGH-13	Free tier Render web + DB (DB supprimée après 90 jours idle)	DevOps	NOT FIXED
D-HIGH-14	autoDeploy: true sur main sans status checks requis	DevOps	NOT FIXED
D-HIGH-15	Pas d'Alembic, pas de table schema_migrations	DevOps	NOT FIXED

ID	Titre court	Axe	Statut
D-HIGH-16	Pas de base PG schema migration (dépendance create_all cachée)	DevOps	NOT FIXED
D-HIGH-17	Actions GitHub non SHA-pinnés (trivy-action@master)	DevOps	NOT FIXED
D-HIGH-18	Logging print() only — pas de Sentry, pas de /metrics, pas de rate limit	DevOps	NOT FIXED
F-HIGH-19	Home.tsx 14 315 lignes — pas de lazy loading → bundle > 2 Mo	Frontend	NOT FIXED
F-HIGH-20	useAuth écrit PII dans localStorage à chaque render	Frontend	NOT FIXED
F-HIGH-21	dev-login endpoint exposé même en non-dev (route enregistrée)	Frontend	NOT FIXED
F-HIGH-22	RBAC client-side uniquement avant /super-admin	Frontend	NOT FIXED
F-HIGH-23	tRPC shim endswith() matching — pas de route exacte	Frontend	NOT FIXED
F-HIGH-24	__manus__/debug-collector.js (28 KB) dans public/	Frontend	NOT FIXED
F-HIGH-25	SuperAdmin.tsx 11 × as any + ligne JSX 7 500 caractères	Frontend	NOT FIXED
F-HIGH-26	0 test page Home/LabBench/SuperAdmin, 0 e2e	Frontend	NOT FIXED
T-HIGH-27	SEC-08 privilège escalation /api/lab/sessions/step-up roles client-supplied	Tests	NOT FIXED
T-HIGH-28	pgserver absent de requirements.txt → 4 tests ERROR en CI	Tests	NOT FIXED
T-HIGH-29	TestClient cookie isolation cassée — test 2-user isolation SKIPPED	Tests	NOT FIXED
T-HIGH-30	validation/*.log cités comme preuves mais répertoire vide	Tests	NOT FIXED
T-HIGH-31	Dernière livraison (29 oct) non validée pytest backend	Tests	NOT FIXED

Tableau 3 — Liste exhaustive des défauts HIGH (31).

Top 5 bloquants production — synthèse visuelle

Le schéma ci-dessous résume la priorisation d’ action. Chaque pastille représente un défaut CRITICAL. Les cinq pastilles en première ligne sont les corrections à appliquer dès le sprint 1 ; leur absence rend l’ archive non déployable.

Action immédiate — Sprint 1 (semaine 1-2)

Appliquer les douze correctifs du chapitre 10 dans l'ordre indiqué. Aucun autre travail fonctionnel ne doit être engagé tant que la branche sprint-1-critical-fixes n'a pas passé la CI (une fois celle-ci durcie) et que la suite pytest + vitest est verte. Ces correctifs représentent environ **3 à 5 jours-homme** de travail concentré.

CHAPITRE 4 · BACKEND PYTHON

Inspection Backend Python — Findings détaillés

Le backend AeroMind est composé de 41 modules Python dans le répertoire `web/`, totalisant environ 14 000 lignes de code. L'architecture est globalement saine — séparation des préoccupations, RBAC à six rôles, modes opérationnels explicites, audit append-only avec correlation_id — mais l'inspection révèle **17 défauts actionnables** dont **4 critiques** qui rendent plusieurs fonctionnalités clés injoignables en l'état. Ce chapitre détaille chaque défaut par sous-axe, avec emplacement précis, impact concret et correctif recommandé.

4.1 Architecture & structure des modules

Les 41 modules se répartissent en cinq grandes familles fonctionnelles : (1) **Application & middleware** — `app.py` (1 666 lignes), `config.py`, `database.py` ; (2) **Authentification** — `oidc.py` (Microsoft Entra), `local_auth.py` (MFA PBKDF2 + TOTP AES-GCM), `labbench_auth.py` (RBAC + step-up + hardware tokens HMAC), `session_store.py`, `app_auth_helpers.py`, `admin.py` ; (3) **LABBENCH canonique** — `canonical_models.py`, `labbench_canonical_api.py` (737 lignes, 12 endpoints), `labbench.py` (1 681 lignes, moteur de simulation) ; (4) **MAVLink & matériel** — `mavlink_pool.py`, `mavlink_command_api.py`, `mavlink_mission_protocol.py`, `mavlink_observer.py`, `mavlink_telemetry_api.py`, `hardware_api.py`, `hardware_gate.py`, `geofence_validator.py`, `mission_planner.py`, `operation_mode.py`, `operational_state.py` ; (5) **Domaine métier** — `business_api.py`, `command_center.py`, `copilot.py`, `engineering_extensions.py`, `fleet_telemetry.py`, `dashboard.py`, `preflight.py`, `maintenance.py`, `material_library.py`, `realtime.py`, `voice_commands.py`, `i18n.py`.

Le graphe de dépendances entre modules présente des **cycles d'importation lazy** autour du triangle `labbench_auth` ↔ `session_store` ↔ `labbench_canonical_api`. Ces trois modules s'importent mutuellement à l'intérieur des fonctions (et non au niveau du module) pour éviter l'erreur `ImportError: cannot import name X (most likely due to a circular import)`. Cette pratique masque les dépendances réelles, complique l'analyse statique et fait qu'une erreur d'import deviendra un HTTP 500 au premier appel plutôt qu'au démarrage du serveur. Une passe de refactor minimaliste (extraire un module `labbench_types` ne contenant que la dataclass `StepUpSession` et les constantes partagées) casserait le cycle sans remettre en cause l'architecture.

Le fichier `web/app.py` concentre à lui seul 1 666 lignes et mélange l'enregistrement des 21 routeurs, le pont de compatibilité tRPC (`trpc_compat`, ~470 lignes avec SQL embarqué), le store des workspaces anonymes, les endpoints d'authentification dev-login et le middleware global. Ce monolithe rend les tests unitaires difficiles et bloque toute refactor non triviale. La priorité de refactor (traitée en sprint 3) est d'extraire le pont tRPC dans

un module dédié `trpc_bridge.py` avec un dispatcher par famille de procédures.

4.2 Couche de données — divergences PostgreSQL / TiDB

L'ORM est déclaré en deux endroits : `web/models.py` (modèle legacy, ~25 tables) et `web/canonical_models.py` (modèle LABBENCH-01, qui alias la table `LabSession` du legacy). La migration PostgreSQL canonique (`migrations/2026-10-25_canonical_labbench_schema.sql`) ajoute des colonnes typées (`assurance_level`, `state`, `expires_at`, `idempotency_key`) que l'ORM ne déclare pas — ces colonnes restent donc NULL en PostgreSQL. À l'inverse, l'ORM écrit systématiquement dans `data_json` (blob JSON).

> **CRITIQUE B-CRIT-06 — Migration TiDB divergente**
>
> La variante TiDB `migrations/2026-10-25_canonical_labbench_schema_tidb.sql` diverge de la variante PostgreSQL en trois points bloquants : (1) la colonne `lab_sessions.id` est `VARCHAR(36)` en TiDB contre `VARCHAR(128)` en PostgreSQL ; or `session_store.py:297` appelle `secrets.token_urlsafe(48)` qui produit ~64 caractères — tout insert échoue avec « Data too long for column id » ; (2) `lab_sessions.data_json` existe en PostgreSQL (line 54) mais **n'existe pas en TiDB**, alors que l'ORM la déclare `nullable=False` — tout insert échoue ; (3) les colonnes `missions.upload_protocol` et `missions.upload_idempotency_key` sont absentes de la variante TiDB, alors que `mission_planner.py:318` les utilise pour le check d'idempotence.

Plus grave encore, le fichier `TIDB_SCHEMA.sql` (48 lignes) ne couvre que 3 tables sur ~30 (`audit_events`, `admin_accounts`, `admin_account_events`). Un opérateur suivant `TIDB_SETUP_FR.md` qui s'appuierait sur ce fichier obtiendrait une base sans `users`, `projects`, `drones`, `lab_sessions`, `missions`, `hardware_tokens`, etc. La ligne de vie `Base.metadata.create_all(bind=engine)` dans `web/app.py:88-90` ne s'exécute que si `AEROMIND_AUTO_CREATE_SCHEMA=true`, ce qui est le cas dans `render.yaml` — mais le DDL SQLAlchemy ne supporte pas les `IF NOT EXISTS` sur certaines contraintes TiDB.

Correctif — synchroniser TiDB avec PostgreSQL

```
--- migrations/2026-10-25_canonical_labbench_schema_tidb.sql
+++ migrations/2026-10-25_canonical_labbench_schema_tidb.sql (corrigé)
@@ -36,1 +36,1 @@
- id VARCHAR(36) NOT NULL PRIMARY KEY,
+ id VARCHAR(128) NOT NULL PRIMARY KEY,
@@ -54,0 +54,1 @@
+ data_json LONGTEXT NOT NULL,
@@ -189,3 +189,3 @@
--- Idempotent guards (TiDB ≥ 7.x ne supporte pas IF NOT EXISTS sur ADD COLUMN)
-ALTER TABLE missions ADD COLUMN upload_state VARCHAR(20) DEFAULT 'pending';
-ALTER TABLE missions ADD COLUMN upload_started_at DATETIME NULL;
-ALTER TABLE missions ADD COLUMN upload_error TEXT NULL;
+-- Procédure idempotente via information_schema
+SET @col_exists = (SELECT COUNT(*) FROM information_schema.columns
+ WHERE table_schema=DATABASE() AND table_name='missions'
+ AND column_name='upload_state');
+SET @sql = IF(@col_exists=0,
+ 'ALTER TABLE missions ADD COLUMN upload_state VARCHAR(20) DEFAULT ''pending''',
+ 'SELECT 1'); PREPARE s FROM @sql; EXECUTE s; DEALLOCATE PREPARE s;
```

```
+-- Idem pour upload_protocol, upload_idempotency_key, upload_started_at, upload_error
```

4.3 Authentification & autorisation

Trois régimes d'authentification cohabitent sans dépendance partagée : (1) **OIDC Microsoft Entra** via `web/oidc.py`, qui pose un cookie `aeromind_session` signé HMAC ; (2) **MFA locale** via `web/local_auth.py` (PBKDF2 pour le mot de passe, AES-GCM pour le secret TOTP chiffré en base) ; (3) **Step-up LABBENCH** via `web/labbench_auth.py` et `web/session_store.py`, qui émet une session à TTL court (15 min) après validation d'un code MFA quotidien (et non, contrairement à ce que dit la doc, un TOTP RFC-6238 par utilisateur — voir finding T-MED-13 sur la contradiction documentation/code). Les anciens endpoints (business_api, mission_planner, hardware_api, preflight, maintenance, dashboard) utilisent directement `current_user(aeromind_session, authorization)`, tandis que les nouveaux endpoints canoniques `/api/lab/*` attendent une session step-up via `verify_lab_session`.

> **CRITIQUE B-CRIT-01 — current_user_or_401 inexistant**
>
> Le endpoint `POST /api/lab/sessions/step-up`, seule porte d'entrée du routeur canonique LABBENCH, exécute :
>
> ```
> from .app_auth_helpers import current_user_or_401
> user = current_user_or_401(request)
> ```
>
> Or le fichier `web/app_auth_helpers.py` (25 lignes) ne définit que `current_user`. À chaque appel, `ImportError: cannot import name 'current_user_or_401'` est levée → HTTP 500. **Les 12 endpoints canoniques /api/lab/* sont donc tous injoignables.** Aucune session step-up ne peut être émise, aucune DroneRevision créée, aucun TestRun démarré, aucune gate G0-G5 approuvée.

Correctif — définir current_user_or_401

```
--- web/app_auth_helpers.py (actuel, 25 lignes)
+++ web/app_auth_helpers.py (corrigé, +6 lignes)
@@ -25,3 +25,9 @@ def current_user(cookie, authorization=None):
if user is None:
return None
return user
+
+def current_user_or_401(request) -> dict:
+ """Return the authenticated user or raise 401 immediately."""
+ user = current_user(
+ request.cookies.get('aeromind_session'),
+ request.headers.get('authorization'),
+ )
+ if user is None:
+ from fastapi import HTTPException
+ raise HTTPException(status_code=401, detail={
+ 'code': 'UNAUTHENTICATED',
+ 'message': 'Authentication required.'
```

```
+ })
+ return user
```

CRITIQUE B-CRIT-02 — HMAC token matériel toujours "insecure-dev-only"

La fonction `_hmac_secret()` à `web/labbench_auth.py:381-383` fait :

```
secret = getattr(settings, "AEROMIND_SESSION_SECRET", "") or "insecure-dev-only"
return secret.encode("utf-8")[:64]
```

Problème : `settings` est le `_SettingsProxy` qui wrap une instance de `Settings`. Pydantic Settings expose l'attribut Python (`session_secret`), pas l'alias d'environnement (`AEROMIND_SESSION_SECRET`). Donc `getattr` retourne toujours la valeur par défaut `""`, et la fonction retombe sur `"insecure-dev-only"`. **Tous les tokens matériels de toutes les instances AeroMind du monde sont signés avec la même clé publique.**

Correctif — _hmac_secret utilise get_settings().session_secret

```
--- web/labbench_auth.py:381-383 (actuel)
+++ web/labbench_auth.py:381-383 (corrigé)
-def _hmac_secret() -> bytes:
- secret = getattr(settings, "AEROMIND_SESSION_SECRET", "") or "insecure-dev-only"
- return secret.encode("utf-8")[:64]
+def _hmac_secret() -> bytes:
+ # Pydantic Settings expose l'attribut Python, pas l'alias env.
+ secret = get_settings().session_secret
+ if not secret or len(secret) < 32:
+ raise RuntimeError(
+ "AEROMIND_SESSION_SECRET must be set and >= 32 bytes "
+ "for hardware token HMAC signing."
+ )
+ return secret.encode("utf-8")[:64]
```

CRITIQUE B-CRIT-03 — CSRF défini mais jamais appliqué

La fonction `verify_csrf_double_submit` est définie à `web/labbench_auth.py:146-165` et exportée dans `__all__` (line 549), mais aucune route ne l'utilise. Le cookie `aeromind_session` est posé avec `samesite='none'` et `secure=True` en production (`web/oidc.py:94`), pour autoriser l'intégration iframe cross-site. Sans enforcement CSRF, **tout endpoint state-changeant est vulnérable à CSRF** : un attaquant sur `evil.com` peut exécuter `fetch('https://victim.onrender.com/api/hardware/drones/X/command', {method:'POST', credentials:'include', ...})` et la requête transportera le cookie de la victime.

Correctif — appliquer Depends(verify_csrf_double_submit) sur toutes les routes mutantes

```
--- web/app.py (routeurs enregistrés, L150-171)
+++ web/app.py (corrigé)
@@ -150,2 +150,8 @@ app.include_router(business_router)
-app.include_router(hardware_router)
-app.include_router(labbench_canonical_router)
+# CSRF obligatoire sur toute route state-changeante.
+_csrf = [Depends(verify_csrf_double_submit)]
```

```
+app.include_router(hardware_router, dependencies=_csrf)
+app.include_router(labbench_canonical_router, dependencies=_csrf)
+app.include_router(mission_planner_router, dependencies=_csrf)
+app.include_router(local_auth_router, dependencies=_csrf)
+app.include_router(oidc_router, dependencies=_csrf)
+app.include_router(admin_router, dependencies=_csrf)
```

4.4 Posture sécurité — endpoints matériel & dashboard

CRITIQUE B-CRIT-04 — /api/hardware/pool/register omet owner_id → 400 systématique

L'appel `pool.register(drone_id, drone_name, connection_url)` à `web/app.py:1202` ne passe pas `owner_id` ni `project_id`. Or `MavlinkPool.register` (`web/mavlink_pool.py:92-114`) lève explicitement :

`HardwareCommandError(400, "BAD_REQUEST", "owner_id et project_id sont requis (SEC-02 isolation inter-tenant)")`

Aucun drone ne peut jamais être enregistré dans le pool. Conséquence en chaîne : `issue_hardware_confirmation_token` échoue toujours (le check `drone_registered_in_pool` n'est jamais vrai), donc **aucune commande MAVLink physique ne peut jamais être envoyée**.

Correctif — passer owner_id et project_id partout

```
--- web/app.py:1202 (actuel)
-entry = pool.register(drone_id, drone_name, connection_url)
+++ web/app.py:1202 (corrigé)
+entry = pool.register(
+ drone_id, drone_name, connection_url,
+ owner_id=user['id'],
+ project_id=project_id or '',
+)

--- web/app.py:1227 (unregister) ---
-mavlink_pool_service.get_pool().unregister(drone_id)
+mavlink_pool_service.get_pool().unregister(drone_id, owner_id=user['id'])

--- web/app.py:1304 (pilot pour command) ---
-pilot = pool.pilot(drone_id)
+pilot = pool.pilot(drone_id, owner_id=user['id'])

--- web/app.py:1427 (set_geofence) ---
-geofence_service.set_geofence(geofence)
+geofence_service.set_geofence(geofence, owner_id=user['id'])
```

HIGH B-HIGH-02 — Dashboard recent_alerts expose les alertes de tous les tenants

La requête `select(AlertEvent).order_by(...).limit(5)` à `web/dashboard.py:113-126` n'est pas scopée par `Project.user_id`. N'importe quel utilisateur authentifié qui appelle `GET /api/dashboard` reçoit les 5 alertes les plus récentes **tous projets et tous utilisateurs confondus**. Idem pour le compteur `activeAlerts` à la ligne 60-65. Violation directe de l'exigence SEC-02 (isolation inter-tenant).

Correctif — scoper les requêtes dashboard par utilisateur

```
--- web/dashboard.py:113-126 (actuel)
-recent_alerts = db.scalars(
- select(AlertEvent)
- .order_by(AlertEvent.created_at.desc())
- .limit(5)
-).all()
+++ web/dashboard.py:113-126 (corrigé)
+from sqlalchemy import select as _select
+_user_projects_subq = _select(Project.id).where(Project.user_id == user['id'])
+recent_alerts = db.scalars(
+ _select(AlertEvent)
+ .where(AlertEvent.project_id.in_(_user_projects_subq))
+ .order_by(AlertEvent.created_at.desc())
+ .limit(5)
+).all()
```

4.5 Intégration MAVLink — pool, telemetry consent, mission upload

Le module `web/mavlink_pool.py` implémente correctement l'isolation inter-tenant au niveau de la couche pool — les méthodes `register`, `entry`, `pilot`, `observer`, `unregister` exigent ou acceptent un `owner_id`. Le problème est que la couche API (`web/app.py`) ne le transmet jamais — voir B-CRIT-04 et B-HIGH-01 ci-dessus. Une fois ces correctifs appliqués, l'isolation sera effective.

HIGH B-HIGH-04 — Telemetry consent en mémoire uniquement

Le dictionnaire `_consent_state` à `web/mavlink_telemetry_api.py:47` est un dict Python par-processus. Un utilisateur accorde le consentement sur le worker A ; le prochain poll tombe sur le worker B (round-robin Render) qui n'a pas l'entrée → 403 « Consentement télémétrie requis ». Multi-travailliste (4+ workers), le consentement doit être re-accordé sur chaque worker, ce qui est impossible. **La révocation souffre du même défaut** : révoquer sur un worker ne révoque pas sur les autres jusqu'à expiration du TTL.

Correctif — persister le consentement en DB

```
--- web/mavlink_telemetry_api.py:47 (actuel)
-_consent_state: dict[str, dict] = {} # par-processus
+++ web/mavlink_telemetry_api.py:47 (corrigé)
+# Persister dans lab_sessions.data_json["consent"] via session_store
+def _get_consent(user_id: str, drone_id: str) -> dict | None:
+ from .session_store import get_session
+ sess = get_session(user_id)
+ if not sess:
```

```
+ return None
+ return sess.data.get('consent', {}).get(drone_id)
```

Le module `web/mission_planner.py` déclare la fonction `_issue_mission_item_token` (lignes 478-496) qui n'est **jamais appelée**. La fonction passe `reviewer_id=actor_id` (line 491) avec un commentaire « NOTE: in real flow, reviewer_id != actor_id (EXIGENCE IAM-03) » — elle violerait donc IAM-03 (operator ≠ approbateur) si elle était appelée. Le code mort qui viole un invariant de sécurité est un bug latent. Correctif : supprimer la fonction ou la câbler correctement dans le path d'upload per-waypoint avec un vrai `reviewer_id`.

4.6 Modes opérationnels & gates G0-G5

Le module `web/operation_mode.py` définit proprement la machine à états DEMO/SIMULATION/SITL/HIL/HARDWARE/OFFLINE via la fonction `determine_mode(...)`. Cependant, deux problèmes : (1) l'état SITL/HIL est lu depuis des variables d'environnement globales (`AEROMIND_SITL_ACTIVE`, `AEROMIND_HIL_ACTIVE`) plutôt qu'une colonne par-projet en base — deux projets ne peuvent pas avoir des modes différents ; (2) les fonctions de garde `assert_no_physical_command` et `assert_persisted_or_demo` (lignes 197-220) sont définies mais **jamais appelées** dans le code. L'exigeance PHY-05 (« aucun déclenchement automatique de moteurs par l'UI ») et SYNC-07 (« impossible d'afficher 'réel' pour une valeur synthétique ») ne sont donc pas enforced.

> **HIGH B-HIGH-06 — Gates G0-G5 non vérifiées dans le path commande matériel**
>
> Le cahier des charges exige C-001 : « Toutes sorties MAVLink matérielles restent bloquées jusqu'à satisfaction documentée des portes G0 à G5 ». En pratique, `authorize_physical_command` (`web/mavlink_command_api.py:90-108`) vérifie `HARDWARE_CONTROL=true`, l'allowlist de commandes, le token de confirmation, le binding de contexte et la géofence — **mais pas les gates G0-G5**. Le path `hardware_gate.issue_hardware_confirmation_token` vérifie seulement N3 armé + hardware review signé + drone registered in pool, qui est un sous-ensemble plus petit que G0-G5. Un opérateur peut donc émettre une commande MAVLink avec seulement N3 + review + pool, sans que les 6 gates G0-G5 ne soient approuvées.

Correctif — vérifier les 6 gates avant d'émettre un token

```
--- web/hardware_gate.py:319-427 (dans issue_hardware_confirmation_token)
+++ web/hardware_gate.py:319-427 (corrigé)
@@ après le check drone_registered_in_pool @@
+# Vérifier les 6 gates G0-G5 (EXIGENCE C-001)
+from .canonical_models import GateValidation
+required_gates = ['G0_gel', 'G1_securite', 'G2_fiabilite',
+ 'G3_banc', 'G4_qualification', 'G5_exploitation']
+approved = {r.gate_name for r in db.scalars(
+ select(GateValidation).where(
+ GateValidation.project_id == project_id,
+ GateValidation.state == 'approved',
+ )
+)}
```

```
+missing = set(required_gates) - approved
+if missing:
+ raise HardwareCommandError(
+ 412, 'GATE_NOT_READY',
+ f'Gates manquantes: {sorted(missing)}'
+ )
```

4.7 Gestion d'erreurs & logging

L'inspection recense **71 occurrences** de `except Exception` à travers `web/*.py`. Ces handlers larges masquent les causes racines, transforment des erreurs DB en échecs silencieux (la fonction retourne `None` et l'appelant interprète cela comme « session non trouvée ») et rendent le débogage production très difficile. Le handler global (`web/app.py:1662-1665`) retourne le nom du type d'exception au client (`type: OperationalError`) — fuite d'information mineure mais réelle. Aucun correlation ID n'est propagé. Aucune trace n'est loggée côté serveur pour les 500.

Le logging applicatif repose presque exclusivement sur des `print(..., flush=True)` (11 occurrences dans `web/app.py`, plus d'autres dans `web/admin.py`, `web/operational_state.py`, etc.). Seuls trois modules (`labbench_auth.py`, `session_store.py`, `mavlink_mission_protocol.py`) utilisent `logging.getLogger`. La configuration racine du logger n'est jamais appelée, donc le niveau par défaut (WARNING) s'applique. Correctif sprint 4 : adopter `structlog` pour du JSON, intégrer `sentry-sdk[fastapi]`, ajouter un middleware correlation ID.

4.8 Configuration & variables d'environnement

Le fichier `.env.example` (71 lignes) documente 27 variables d'environnement. L'audit recense **8 variables critiques réellement utilisées par le code mais absentes de .env.example** : `AEROMIND_SITL_ACTIVE`, `AEROMIND_HIL_ACTIVE`, `AEROMIND_AUTO_CREATE_SCHEMA`, `AEROMIND_WORKSPACE_TTL_SECONDS`, `AEROMIND_MISSION_REQUEST_TIMEOUT_S`, `AEROMIND_MISSION_ACK_TIMEOUT_S`, `AEROMIND_DB_CA`, `AEROMIND_MAVLINK_OBSERVER_URL`. L'absence de `AEROMIND_AUTO_CREATE_SCHEMA` dans .env.example fait qu'un déploiement PostgreSQL frais démarre sans tables — le lifespan le lit mais l'opérateur n'a pas la variable documentée.

La valeur par défaut de `AEROMIND_SUPERADMIN_EMAILS` dans `web/config.py:55` est `'mcgregoralvin14@gmail.com'` — une adresse Gmail personnelle. Idem pour `AEROMIND_TRIAL_EMAILS` (line 59) et `DEFAULT_DEMO_EMAIL` (line 62). `web/admin.py:65` conserve `preferred = "mcgregoralvin14@gmail.com"`. Tout déploiement Render qui n'écrase pas ces variables accorde les droits super-admin à cette adresse. Le test `tests/test_demo_email_config.py:8-9` valide que cette adresse est acceptée. **La note du README § « Premier administrateur » qui prétend que l'adresse a été supprimée (SEC-05 fix) est fausse.**

4.9 Qualité code & dette technique

> **HIGH B-HIGH-08 — tRPC bridge async + sync SQLAlchemy → event loop bloqué**
>
> Le handler `trpc_compat` à `web/app.py:473-945` est déclaré `async def` mais appelle directement `SessionLocal()` (SQLAlchemy synchrone). Chaque appel tRPC bloque l'event loop pendant la durée de la requête DB. FastAPI exécute les routes async sur l'event loop principal, donc une requête lente (ou l'épuisement du pool PG) bloque **toutes les requêtes concurrentes**, y compris les heartbeats WebSocket. Avec 50+ utilisateurs concurrents, l'API entière devient inresponsive.

Correctif — déclarer trpc_compat en def synchrone (threadpool)

```
--- web/app.py:473 (actuel)
-@app.api_route('/api/trpc/{procedure:path}', methods=['GET', 'POST'])
-async def trpc_compat(procedure: str, request: Request):
- # ... SessionLocal() bloquant l'event loop
+++ web/app.py:473 (corrigé)
+@app.api_route('/api/trpc/{procedure:path}', methods=['GET', 'POST'])
+def trpc_compat(procedure: str, request: Request):
+ # FastAPI exécute les routes 'def' (non async) dans un threadpool.
+ # L'event loop n'est plus bloqué par les requêtes DB synchrones.
```

4.10 Dépendances Python

Le fichier `requirements.txt` (13 lignes) épingle toutes les versions avec `==` — pratique correcte pour la reproductibilité. Cependant : (1) la version `fastapi==0.141.1` n'existe pas sur PyPI à date — la dernière version publiée fin 2024 est 0.115.x ; il faudra vérifier que la version résout effectivement au déploiement ; (2) `pydantic==2.13.5` et `SQLAlchemy==2.0.44` sont également en avance sur les dernières versions stables connues ; (3) aucun `pip-audit` bloquant n'est configuré en CI (voir D-CRIT-09). Correctif sprint 2 : ajouter un step `pip install --dry-run -r requirements.txt` en CI pour vérifier la résolution, et configurer Dependabot.

4.11 Readiness déploiement — Dockerfile & render.yaml

Le Dockerfile est correctement structuré en multi-stage (stage `frontend-build` avec node:22-bookworm-slim → stage `runtime` avec python:3.12-slim), crée un utilisateur non-root `aeromind`, et active `PIP_NO_CACHE_DIR`, `PYTHONDONTWRITEBYTECODE`, `PYTHONUNBUFFERED`. Trois améliorations : (1) le `CMD` utilise la forme shell (`CMD ["sh", "-c", "uvicorn ... ${PORT:-10000}"]`) qui exécute uvicorn comme enfant de `sh` PID 1 — le SIGTERM de Render prend ~10s à se propager ; préférer la forme exec ; (2) aucune directive `HEALTHCHECK` dans l'image (Render utilise `healthCheckPath` externe, mais tout autre environnement — local docker, k8s, fly.io — n'aura pas de health status) ; (3) le `.dockerignore` (15 lignes) n'exclut pas `docs/`, `engineering/`, `tests/`, `*.md`, `AUDIT_I18N_*.json` (~2.5 MB), `.github/`, ce qui alourdit le contexte de build.

> **HIGH B-HIGH-09 — render.yaml AUTO_CREATE_SCHEMA=true en production**

> Le Blueprint Render `render.yaml:33-34` positionne `AEROMIND_AUTO_CREATE_SCHEMA: "true"` en production. Or la doc `DEPLOYMENT_RENDER_ENVIRONMENT.md:30` et `TIDB_SETUP_FR.md:38-43` stipulent explicitement `false` en production (« le compte applicatif ne doit pas avoir besoin du privilège CREATE pendant chaque redémarrage »). Avec true, chaque cold start exécute `Base.metadata.create_all()` — ce qui crée les tables depuis les déclarations ORM, sans les contraintes CHECK, FK, et index de la migration canonique. Après le premier déploiement, toute évolution ORM (ajout de colonne) est silencieusement no-op car `create_all` ne fait pas `ALTER TABLE`.

Correctif — render.yaml corrigé + preDeployCommand

```
--- render.yaml:33-34 (actuel)
-environmentVars:
- AEROMIND_AUTO_CREATE_SCHEMA: "true"
+++ render.yaml:33-34 (corrigé)
+environmentVars:
+ AEROMIND_AUTO_CREATE_SCHEMA: "false"
+preDeployCommand: "bash scripts/apply_migrations.sh $DATABASE_URL"

# scripts/apply_migrations.sh (à créer)
# #!/usr/bin/env bash
# set -euo pipefail
# psql "$1" -v ON_ERROR_STOP=1 \
# -f migrations/2026-10-21_admin_account_registry.sql \
# -f migrations/2026-10-21_validation_audit_events_pg.sql \
# -f migrations/2026-10-25_canonical_labbench_schema.sql
```

CHAPITRE 5 · FRONTEND REACT

Inspection Frontend React — Findings détaillés

Le frontend AeroMind est une application React 19.2.1 / Vite 7.1.7 / TypeScript 5.9.3 avec tRPC client 11.6 (dont le contrat `AppRouter` est déclaré `any` — voir F-CRIT-11), @tanstack/react-query 5.90, Tailwind 4, Three.js 0.186 + R3F 9 pour le jumeau mécanique, et wouter pour le routing. Le code source représente environ 43 000 lignes TypeScript/TSX réparties sur ~80 composants et pages. L'inspection révèle **24 défauts actionnables** dont **3 critiques** qui exposent directement la sécurité des utilisateurs.

5.1 Architecture & structure du projet

CRITIQUE F-CRIT-11 — tRPC AppRouter = any

Le fichier `frontend/server/routers.ts` ne contient que deux lignes :

```
export type AppRouter = any;
```

Côté client, `frontend/client/src/lib/trpc.ts:7` fait :

```
createTRPCReact() as any
```

Conséquence : **chaque appel** `trpc...useQuery/useMutation` (useAuth.ts:19, LabBench.tsx × 14 sites, Home.tsx × 12 sites, SuperAdmin.tsx:51-61) perd toute vérification de type input/output. Un renommage de procédure backend casse le runtime, pas la compilation. Le répertoire `frontend/server/` est un poids mort trompeur.

Correctif — remplacer par un fetchTRPC typé depuis OpenAPI

```
// frontend/client/src/lib/api.ts (nouveau, typé depuis FastAPI OpenAPI)
import type { paths } from './generated/openapi-types';

type FetchOpts = { method?: string; body?: unknown };
export async function apiFetch(
path: P, method: M, body?: paths[P][M] extends { requestBody: { content: { 'application/json': infer B }
} } ? B : never
): Promise {
const res = await fetch(path, {
method: method as string,
credentials: 'include',
headers: { 'Content-Type': 'application/json', 'X-AeroMind-Language': currentAppLanguage() },
body: body !== undefined ? JSON.stringify(body) : undefined,
});
if (!res.ok) throw new ApiError(res.status, await res.json().catch(() => ({})));
return res.json();
}
```

```
// Puis remplacer tous les trpc.x.y.useQuery() par des hooks React Query typés
// générés depuis l'OpenAPI : useApiQuery('/api/projects', 'get')
```

Le fichier `frontend/client/src/pages/Home.tsx` totalise **14 315 lignes** — le plus gros fichier du projet, environ 33 % du code frontend total. Il est importé de façon synchrone dans `App.tsx:9`, ce qui annule toute stratégie de `manualChunks` : Three.js, recharts, framer-motion, hls.js et Streamdown sont dans le chemin critique de la première page. Le commentaire à la ligne 8 du fichier reconnaît le problème (« Home.tsx is 14k lines ») mais le fix — lazy loading — n'est appliqué qu'à `SuperAdmin`. La taille du fichier a augmenté de 12 956 à 14 315 lignes depuis l'audit du 14 octobre, soit +10 %, en contradiction totale avec la recommandation P2-01 du rapport AUDIT_TECHNIQUE_AEROMIND_2026-10-14 qui préconisait l'éclatement.

5.2 Configuration de build

Le fichier `frontend/package.json` déclare deux devDependencies suspectes : `"add": "^2.0.6"` (ligne 103) et `"pnpm": "10.18.0"` (ligne 107). Le package `add` est un utilitaire no-op qui ne devrait jamais apparaître dans une application de production — c'est presque certainement le résultat d'une commande `pnpm add` malformée. Déclarer pnpm comme devDependency fait que le build Docker télécharge pnpm deux fois (via corepack et via le graphe d'install). Le pinning effectif se fait via `"packageManager": "pnpm@10.18.0"` (ligne 117) et `corepack prepare pnpm@10.18.0 --activate` dans le Dockerfile — supprimer les deux entrées superflues.

Le `tsconfig.json` active `"strict": true` mais désactive immédiatement `"noImplicitAny": false` (ligne 11), affaiblissant la garantie. Le fichier exclut également les fichiers de test de la vérification de types (`"exclude": [..., "**/*.test.ts"]`). Le codebase compte au moins 20 casts explicites `: any` dans `SuperAdmin.tsx` et `LabBench.tsx` seuls. Le `vite.config.ts:47` désactive les source maps en production (`sourcemap: false`), ce qui rend le débug des erreurs minifiées impossible — l'ErrorBoundary.tsx:38 affiche `error.stack` qui sera du charabia sans source maps.

Le `vite.config.ts:40` conserve un host de preview Manus (`"4173-iiu0bwsf8nrsan73hz6bs-1c1fd84a.us3.manus.computer"`) qui devrait être retiré. Le `Dockerfile:12` fait `COPY scripts ../scripts` — un chemin atypique en dehors du `WORKDIR` — qui ne fonctionne que parce que pnpm préserve le symlink vers le parent.

5.3 Routing & pages

L'application expose 5 routes via `manus-routes.json` : `/`, `/labbench`, `/admin`, `/super-admin`, `/404`. Les deux routes `/admin` et `/super-admin` montent le même composant — l'alias `/admin` est conservé pour compatibilité mais crée une ambiguïté dans les logs d'audit. Il n'existe pas de routes dédiées pour `/engineering`, `/simulation`, `/mission-planner` : chaque écran est un onglet à l'intérieur de `Home.tsx`.

> Le service worker `frontend/client/public/sw.js` (125 lignes) est **code mort** : aucun appel `navigator.serviceWorker.register("/sw.js")` n'existe dans le code. Il définit un cache app-shell et un cache network-first pour `/api/*` — s'il était enregistré naïvement, il cacherait des réponses authentifiées et provoquerait une fuite de données utilisateur entre sessions sur appareils partagés. Le manifest PWA `manifest.json:5` déclare `"start_url": "/labbench"` ce qui crée une confusion dans les analytics (installations PWA lancées sur `/labbench` alias de `/`).

5.4 Inventaire des composants

Les 10 plus gros composants représentent à eux seuls la majorité du code frontend :

Composant	Lignes	Rôle
pages/Home.tsx	14 315	Coquille principale — cockpit multi-onglets
pages/LabBench.tsx	4 364	Module LABBENCH 01 — 21 vues inline
DroneSketchProWorkbench.tsx	2 490	Atelier de dessin drone
DesignStudioView.tsx	2 170	Studio de design
IDDSTwinAssemblyEditor.tsx	1 741	Éditeur jumeau assemblage IDDS
DigitalTwinExplorer.tsx	1 481	Explorateur jumeau numérique
pages/ComponentShowcase.tsx	1 437	Showcase (MORT — pas importé)
AeroLabPanel.tsx	1 412	Panneau aéro-lab
MechanicalTwinEditor.tsx	1 243	Éditeur jumeau mécanique — 7 nav sections
FleetRadar.tsx	720	Radar flotte

Tableau 4 — Top 10 composants par taille.

L'utilisation de Three.js / R3F dans `MechanicalTwinViewport.tsx` est l'une des rares parties **bien engineerée** du codebase : la sonde WebGL2 (`canCreateWebGL2Context` à `PhysicalDroneModel.tsx:35-54`) libère correctement le contexte de test via `WEBGL_lose_context`, un plafond de 4 canvases WebGL actifs (`MAX_ACTIVE_WEBGL_CANVASES`) avec une file d'attente `reserveWebGLCanvas`. La fuite de contexte WebGL est explicitement évitée.

Le composant `OpenStreetMapView.tsx` implémente sa propre projection Mercator, son propre template d'URL de tuiles, son propre pan/zoom au lieu d'utiliser `maplibre-gl` (MIT). La liste des providers de tuiles (Esri, OSM, CARTO) est codée en dur. Conséquences : pas de tuiles vectorielles, pas de pinch-zoom tactile, pas de contrôle d'attribution — **risque légal avec les tuiles OSM (ODbL exige attribution visible)**. Adoption de maplibre-gl recommandée en sprint 4.

5.5 Gestion d'état

> **HIGH F-HIGH-20 — useAuth écrit PII dans localStorage à chaque render**
>
> Le hook `useAuth.ts:57-72` exécute :
>
> `const state = useMemo(() => { localStorage.setItem("manus-runtime-user-info", JSON.stringify(meQuery.data)); return {...}; }, [meQuery.data, ...]);`
>
> Trois problèmes : (1) `localStorage.setItem` est un effet de bord à l'intérieur d'un `useMemo` — React 19 strict mode double-invoque les bodies de memo en dev, ce qui double les writes ; (2) l'objet complet `meQuery.data` (email, nom, rôle, niveau, statut du compte — toutes PII) est persisté non chiffré dans localStorage sous la clé `manus-runtime-user-info`, accessible à tout XSS ; (3) au logout (ligne 47-49), les clés `aeromind-blank-workspace-v3`, `aeromind-workbench-capture`, `WORKSPACE_KEY_STORAGE_KEY` sont supprimées, **mais pas** `manus-runtime-user-info` — l'identité utilisateur survit au logout.

Correctif — déplacer dans useEffect + cleanup au logout

```
// frontend/client/src/_core/hooks/useAuth.ts (corrigé)
-const state = useMemo(() => {
- localStorage.setItem("manus-runtime-user-info", JSON.stringify(meQuery.data));
- return { user: meQuery.data ?? null, ... };
-}, [meQuery.data, meQuery.error, meQuery.isLoading]);
+const state = useMemo(
+ () => ({ user: meQuery.data ?? null, ... }),
+ [meQuery.data, meQuery.error, meQuery.isLoading],
+);
+useEffect(() => {
+ if (meQuery.data) {
+ localStorage.setItem("manus-runtime-user-info", JSON.stringify(meQuery.data));
+ } else {
+ localStorage.removeItem("manus-runtime-user-info");
+ }
+}, [meQuery.data]);
+// Dans logout():
+localStorage.removeItem("manus-runtime-user-info"); // AJOUTER
```

Le type `ConceptConfig` (`frontend/client/src/lib/aeroEngine.ts:252-277`) est un fourre-tout de 30+ champs (wingspan, fuselageLength, chord, propDiameter, motorCount, mechanicalTwin, mechanicalTwins dict, studioLayers, activeStudioLayer, studioLayerSettings, studioLayerAssignments, renderVariants, customSketch, engineeringSelection, platformProfile, etc.). Il est passé en prop drilling à 6 composants. Chaque `setConfig(mutate)` déclenche un re-render complet de l'arbre workspace. Le `apply()` de `MechanicalTwinEditor.tsx:281` est appelé depuis ~30 handlers. Migration vers un store Zustand (ou `useSyncExternalStore`) recommandée en sprint 4.

5.6 Flux d'authentification

> **CRITIQUE F-CRIT-12 — Session token OIDC mirroré en sessionStorage**

> Le fichier `frontend/client/src/lib/sessionToken.ts:1-47` expose :
>
> `const SESSION_TOKEN_KEY = "aeromind-entra-session";`
> Le token de session est extrait de l'URL hash ou query (post-callback OIDC) et écrit dans `sessionStorage`.
>
> Le `main.tsx:52-73` lit ce token et l'ajoute à toutes les requêtes en `Authorization: Bearer` . Il y a aussi un fallback qui lit `sessionStorage["manus-cookie"]`, le parse comme chaîne de cookies, et forward la valeur `webdev_app_session` comme Bearer token (lignes 59-68).
>
> **Le token de session est lisible par tout JavaScript sur la même origine** (XSS exfiltration). Le fallback `manus-cookie` copie littéralement la valeur du cookie httpOnly dans sessionStorage, **défaillant le but de httpOnly**.

Correctif — supprimer sessionToken.ts, s'appuyer uniquement sur le cookie httpOnly

```
// SUPPRIMER frontend/client/src/lib/sessionToken.ts
// SUPPRIMER les références dans main.tsx:52-73
// Garder uniquement le cookie httpOnly `aeromind_session` posé par web/app.py:355
// main.tsx (corrigé)
-httpLink: httpLink({
- url: '/api/trpc',
- headers: async () => {
- const token = getEntraSessionToken();
- return token ? { authorization: `Bearer ${token}` } : {};
- },
-}),
+httpLink: httpLink({
+ url: '/api/trpc',
+ // credentials: 'include' (default) — le cookie httpOnly suffit.
+}),
```

L'endpoint `/api/auth/dev-login` (`web/app.py:331-357`) est correctement gated par `settings.environment != 'development'` et retourne 403 `DEV_LOGIN_FORBIDDEN` en production — bonne pratique. Cependant la route est **toujours enregistrée** même en production, ce qui la rend découvrable. Une fuite de configuration (par exemple `AEROMIND_ENV=development` par erreur en staging) donnerait à n'importe qui ayant accès réseau un login super_admin. Correctif sprint 4 : n'enregistrer la route que si `environment == 'development'`.

5.7 Internationalisation — deux systèmes parallèles

Le frontend embarque **deux systèmes i18n parallèles**, chargés simultanément dans `LanguageContext.tsx:2,48` : (1) un dictionnaire « français comme clé » de 5 641 entrées × 2 langues (`lib/fr.json`, `lib/en.json`) avec traduction DOM mutation via `translateDom` ; (2) un catalogue « clés namespacées » de 2 617 entrées × 2 langues (`i18n/locales/fr.json`, `i18n/locales/en.json`) avec `resolveMessage`. Le bundle final envoie environ 16 400 chaînes au client.

Sur les 2 617 clés du catalogue namespaced, **2 607 (99,6 %) sont dans le namespace `legacy.*`** avec des clés synthétiques comme `msg2186`. La migration vers les namespaces (`mechanicalTwin.*`, `labBench.gates.*`, etc.) est donc nominale mais pas réelle. Par ailleurs, 183 des 5 641 entrées du dictionnaire ont `fr === en` — pas de traduction réelle. Le rapport `RAPPORT_CLOTURE_I18N_2026-10-16.md` le reconnaît explicitement : « Cette livraison ne constitue pas une relecture linguistique humaine officielle. Les textes ont été générés automatiquement. »

Le `translateDom` est encore plus problématique : il est appelé via un `MutationObserver` sur `document.body` (`LanguageContext.tsx:40-43`), ce qui le déclenche à chaque mutation DOM — chaque ouverture de tooltip, chaque switch d'onglet, chaque `useState`. Sur `Home.tsx` de 14k lignes, c'est un coût de paint mesurable. Correctif sprint 5 : migrer vers des composants explicites et supprimer translateDom + les fichiers `lib/fr.json` et `lib/en.json`.

5.8 Client API & tRPC

> **HIGH F-HIGH-23 — tRPC shim endswith() matching**
>
> Le handler `trpc_compat` à `web/app.py:473-549+` parse la procédure (par exemple `labBench.runSimulation`) puis dispatch via une chaîne de `if name.endswith(...)` (lignes 514, 516, 518, 520, 522, 524, 529, 534, 541, 548).
>
> Conséquence : une requête à `foo.labBench.runSimulation` routerait de la même façon que `labBench.runSimulation` — pas de garde d'exact match. Le transformer SuperJSON sérialise les inputs comme `{json: ...}` mais le shim parse avec un fallback `payload.get('json', raw_payload)` qui peut causer des mismatches silencieux.

Aucune stratégie globale de retry n'est configurée sur le `httpLink` (`main.tsx:47-82`). Les overrides par query sont inconsistants : `retry: false` (useAuth.ts:20), `retry: 1` (LabBench.tsx:669 auth.me, labStatus.tsx:75), `retry: 2, retryDelay: 2000` (LabBench.tsx:675-676 projects.list), `retry: false` par défaut dans Home.tsx et SuperAdmin.tsx. Le défaut React Query est `retry: 3` avec backoff exponentiel — sans override, une 503 sur `auth.me` retry 3 fois sur Home mais 1 fois sur LabBench, créant une UX inconsistante. Correctif : définir un default global `retry: 1, retryDelay: (n) => Math.min(1000 * 2 ** n, 8000)` et n'override que si justifié.

Aucune clé d'idempotency n'est envoyée par le frontend sur les mutations (`trpc.labBench.armN3.useMutation` à LabBench.tsx:975, 1000, 1106, 1131, etc.). Le serveur `web/labbench_auth.py:285-292` supporte pourtant un `lookup_by_idempotency(idempotency_key)` — le contrat existe mais n'est pas utilisé. Un double-clic sur « Arm N3 » peut donc déclencher deux arm commands. Correctif : générer `crypto.randomUUID()` au site d'appel et le passer comme `idempotency_key`.

5.9 UI MAVLink & télémétrie

Le composant `MavlinkTelemetryPanel.tsx:29-43` interroge `/api/telemetry/mavlink/status` toutes les 2 secondes indéfiniment via `setInterval(poll, 2000)`, sans pause quand l'onglet est caché (`visibilitychange` non écouté), sans backoff sur erreur. Sur une mission longue (8 heures), cela représente 14 400 requêtes par opérateur. Le parsing `parseMavlinkObserverSnapshot` (`lib/mavlinkBridge.ts:25-60`) est en revanche **exemplaire** : lignes 27-28 il rejette explicitement le snapshot si `data.read_only !== true || data.hardware_commands_enabled !== false` — parsing défensif correct, refus d'ingérer si le backend venait à envoyer `hardware_commands_enabled: true`.

5.10 Jumeau mécanique / CAD

Le composant `MechanicalTwinEditor.tsx` (1 243 lignes) gère 7 nav sections (Mechanical Twin, Conception, Simulation, Analyse, Fabrication, Données, Collaboration) dans un seul `useState`. La fonction `apply()` (ligne 281) est appelée depuis ~30 handlers. Les révisions du jumeau mécanique sont stockées **uniquement en localStorage** (clé `aeromind-mechanical-revisions-v1:${designId}`), pas sur le serveur. Pour un outil d'ingénierie, c'est inacceptable — les opérateurs attendent une piste d'audit serveur. Le screenshot du viewport (`aeromind-workbench-capture`) est aussi en localStorage. Le `MechanicalTwinViewport.tsx:103-119` reconstruit les BufferGeometry Three.js à chaque changement de prop `explosion` (slider 0..1) — glisser le slider alloue des vertices à 60 fps. Correctif : pré-construire la géométrie une fois et appliquer l'offset en uniform shader.

5.11 Module LabBench 01

Le fichier `pages/LabBench.tsx` (4 364 lignes) définit `ViewId` (21 valeurs, lignes 87-109), `ModeId`, `ZoneId`, `GateState`, `GateDefinition`, puis implémente les 21 vues inline. La progression des gates (N1 read-only → N2 simulation → N3 hardware → N4 sign-off) est correctement modélisée via `gateStateQuery` (ligne 723) et `armN3Mutation` (ligne 975). L'architecture (5 zones, progression des gates) est saine mais l'implémentation monolithique rend tout tweak UI risqué — un changement dans une vue peut casser les autres. Correctif sprint 3 : extraire chaque `ViewId` dans `components/labbench/.tsx`.

5.12 Sécurité client-side

> **CRITIQUE F-CRIT-12 (rappel) — Pas de Content-Security-Policy header**
>
> Aucun en-tête `Content-Security-Policy` n'est posé par le backend (vérifié : `rg -n "Content-Security-Policy" web/` retourne vide). Le middleware `security_headers` (`web/app.py:174-183`) pose bien `X-Content-Type-Options`, `X-Frame-Options`, `Referrer-Policy`, `Permissions-Policy`, `Strict-Transport-Security` — mais **pas CSP**. Combiné avec F-CRIT-12 (token en sessionStorage), n'importe quel XSS = takeover complet de compte. Aucune directive `frame-ancestors` → risque de clickjacking. Aucune restriction `connect-src` → exfiltration vers une origine attaquante non restreinte.

Correctif — ajouter CSP dans le middleware security_headers

```
--- web/app.py:174-183 (actuel, security_headers middleware)
@app.middleware('http')
async def security_headers(request: Request, call_next):
response = await call_next(request)
response.headers['X-Content-Type-Options'] = 'nosniff'
response.headers['X-Frame-Options'] = 'DENY'
response.headers['Referrer-Policy'] = 'strict-origin-when-cross-origin'
response.headers['Permissions-Policy'] = 'geolocation=(), microphone=(), camera=()'
if settings.environment == 'production':
response.headers['Strict-Transport-Security'] = 'max-age=31536000; includeSubDomains'
return response
+++ web/app.py:174-183 (corrigé, +CSP)
@app.middleware('http')
async def security_headers(request: Request, call_next):
response = await call_next(request)
response.headers['X-Content-Type-Options'] = 'nosniff'
response.headers['X-Frame-Options'] = 'DENY'
response.headers['Referrer-Policy'] = 'strict-origin-when-cross-origin'
response.headers['Permissions-Policy'] = 'geolocation=(), microphone=(), camera=()'
if settings.environment == 'production':
response.headers['Strict-Transport-Security'] = 'max-age=31536000; includeSubDomains; preload'
# Content-Security-Policy (tighten progressively after audit)
response.headers['Content-Security-Policy'] = (
"default-src 'self'; "
"script-src 'self'; "
"style-src 'self' 'unsafe-inline'; "
"img-src 'self' data: blob: https:; "
"connect-src 'self' https://login.microsoftonline.com https://api.openai.com "
"wss://*.onrender.com https://*.openstreetmap.org https://server.arcgisonline.com; "
"frame-ancestors 'none'; "
"base-uri 'self'"
)
return response
```

HIGH F-HIGH-24 — `__manus__/debug-collector.js` (28 KB) dans public/

Le script `frontend/client/public/__manus__/debug-collector.js` (900 lignes, 28 175 octets) capture `console.log`, les requêtes `fetch`/XHR et les « semantic uiEvents: click/type/submit/nav/scroll » puis POST périodiquement vers `/__manus__/logs`. **Heureusement, aucune balise <script> ne le charge** dans `index.html` — c'est code mort issu du sandbox preview Manus. Mais sa présence dans le build est un red flag de conformité pour tout audit externe.

Bonnes pratiques relevées côté sécurité : (1) aucune utilisation de `dangerouslySetInnerHTML` — vérifié par `rg -n dangerouslySetInnerHTML frontend/` qui retourne vide. Le linter `lint_i18n_hardcoded.mjs:35-37` l'interdit explicitement. (2) Le cookie OAuth state utilise le préfixe `__Host-` (`frontend/shared/const.ts:11`) qui force `Secure`, `Path=/`, pas de `Domain`. Le nonce est vérifié au callback.

5.13 Accessibilité & UX

Le viewport `index.html:6-8` déclare . Le `maximum-scale=1` viole WCAG 1.4.4 (Resize text) — les utilisateurs malvoyants ne peuvent pas zoomer la page sur mobile. Lighthouse

accessibility audit le signalera. Le `ErrorBoundary.tsx:36-40` affiche `{this.state.error?.stack}` — stack trace complet montré en production (divulgation d'information). Sur `SuperAdmin.tsx:106`, la classe Tailwind est cassée : `xl:grid-cols-inmax(0,1fr)_320px]` — il manque le `[` ouvrant et le `m` de `minmax`. Sur écran xl (⩾1280px), la grille 2 colonnes n'est pas appliquée — régression visuelle passée inaperçue.

5.14 Qualité de code

> **HIGH F-HIGH-25 — SuperAdmin.tsx 11 × as any + ligne JSX 7 500 caractères**
>
> Le fichier `frontend/client/src/pages/SuperAdmin.tsx` cumule plusieurs anti-patterns : 11 casts `as any` (lignes 55-60, 87, 100, 106-107, 114, 117, 118, 120, 148) qui désactivent la sécurité de types. 6 mutations + 3 queries déclarées inline. La ligne 106 est un JSX de ~7 500 caractères contenant toute la table des comptes avec 5 handlers inline (`window.confirm`/`update.mutate`/etc). Diff impossible sur cette ligne. Code review effectivement bloqué.

Le frontend compte **40+ blocs `catch {}` silencieux** (`LanguageContext.tsx:34`, `sessionToken.ts:30`, `useAuth.ts:50`, etc.). Si un `localStorage.setItem` lève `QuotaExceededError`, le draft utilisateur est perdu sans notification. Correctif minimal : `console.warn` l'erreur pour qu'elle apparaisse en dev tools ; pour les échecs user-facing, afficher un toast.

5.15 Tests frontend — couverture critique

> **HIGH F-HIGH-26 — 0 test pour Home.tsx (14 315 lignes), LabBench.tsx (4 364), SuperAdmin.tsx**
>
> L'inventaire des tests frontend recense 57 fichiers `*.test.ts(x)` contenant ~339 tests — **tous dans `lib/` ou `components/`**. Aucun test pour les trois pages les plus grosses et les plus user-facing : `Home.tsx` (14 315 lignes), `LabBench.tsx` (4 364 lignes), `SuperAdmin.tsx`. Aucun test e2e (pas de Playwright, pas de Cypress, pas d'`agent-browser`). L'i18n gate et `pnpm test` passent en CI, mais ils ne testent que des fonctions pures. Toute régression UI dans la progression des gates, le mission planner ou le cycle de vie admin passe inaperçue.

Le Vitest ne définit pas de seuil de couverture : pas de `coverage` dans `package.json` ou `vite.config.ts`. La CI exécute `pnpm test -- --run --reporter=verbose` sans `--coverage`. Correctif sprint 5 : ajouter `vitest --coverage` avec thresholds `{ lines: 60, functions: 60, branches: 50, statements: 60 }` et fail CI en dessous.

5.16 Build readiness

Le build chain est correct : `pnpm run check` (tsc --noEmit + i18n catalogs) puis `pnpm run build` (vite build). Le lockfile est committé, `pnpm install --frozen-lockfile` est appliqué en CI et Docker. Les chunks vendors sont bien découpés (`vendor-three`, `vendor-markdown`, `vendor-hls`, `vendor-ui`, `vendor-radix`, `vendor-data`) via `manualChunks` dans

`vite.config.ts:48-84`. **Mais l’ effet est annulé** car `Home.tsx` est importé de façon synchrone dans `App.tsx:9` et importe lui-même Three.js, recharts, framer-motion au module scope. Le bundle initial dépasse 2 Mo minifié. Correctif sprint 3 : lazy-load Home dans App.tsx et lazy-load les sous-vues lourdes dans Home.

Correctif — lazy load Home et ses sous-vues lourdes

```
// frontend/client/src/App.tsx (corrigé)
-import Home from "@/pages/Home";
+import { lazy, Suspense } from 'react';
+const Home = lazy(() => import("@/pages/Home"));

// Dans le JSX :
-
+}>

// Dans Home.tsx (corrigé) — lazy load des composants lourds
-import MechanicalTwinEditor from '@/components/MechanicalTwinEditor';
-import DroneSketchProWorkbench from '@/components/DroneSketchProWorkbench';
+const MechanicalTwinEditor = lazy(() => import('@/components/MechanicalTwinEditor'));
+const DroneSketchProWorkbench = lazy(() => import('@/components/DroneSketchProWorkbench'));
+// Idem pour DesignStudioView, DigitalTwinExplorer, IDDSTwinAssemblyEditor, AIChatBox
```

CHAPITRE 6 · TESTS & AUDITS

Inspection Tests & Historique d'audit

AeroMind dispose d'une suite de tests backend solide (30 fichiers pytest, ~210 tests) et d'une suite frontend raisonnable (57 fichiers vitest, ~339 tests sur fonctions pures). Cependant, l'inspection révèle **13 défauts actionnables** dont **1 critique** (SEC-08 privilege escalation) et **5 hauts**. Plus préoccupant : la comparaison avec les 75 rapports d'audit accumulés depuis le 13 octobre 2026 révèle des contradictions majeures entre les déclarations de closure et la réalité du code, et plusieurs défauts récurrents identifiés à 4-8 reprises sans correction effective.

6.1 Inventaire des tests

Backend pytest : 30 fichiers sous `tests/`, environ 7 730 lignes, ~210 fonctions de test dont ~12 marquées `pg_only` (skip sur SQLite, exécution sur PostgreSQL). Les plus volumineux : `test_labbench_reception.py` (22 tests, matrice T01-T18 du cahier des charges § 24), `test_hardware_phases.py` (37 tests, MAVLink pilot/pool/geofence/gate N4), `test_voice_commands.py` (17, parser FR/EN), `test_material_library.py` (14, non-weaponized enforcement), `test_lot_d_sessions_csrf.py` (15, R03 TOTP / R04 session / R08 isolation), `test_mavlink_mission_protocol.py` (12, MISSION_COUNT → ACK), `test_fleet_telemetry.py` (12, fleet radar + geofence XY), `test_migrations.py` (10, dont 2 pg_only), `test_workspace_persistence.py` (9), `test_functional_regressions.py` (8, dont 1 skip), `test_labbench_gates.py` (8, pure gate-state progression).

Frontend vitest : 57 fichiers `*.test.ts(x)`, ~339 tests, exclusivement dans `lib/` et `components/`. Top countés : `conceptGenerator.test.ts` (20), `partCommands.test.ts` (16), `anonymousWorkspaceKey.test.ts` (16), `simulationOnlyAdapter.test.ts` (14), `mechanicalTwinGeometry.test.ts` (11).

6.2 Couverture backend — modules sans test dédié

Sept modules backend n'ont aucun test dédié et ne sont pas exercés de manière significative par les tests existants :

Module	Raison	Risque
web/copilot.py	Appel OpenAI non testé	Erreur 500 silencieuse si clé invalide
web/realtime.py	WebSocket send errors swallow	Perte de messages temps réel
web/engineering_extensions.py	NACA aero estimation non testé	Valeurs aberrantes acceptées
web/engineering_document.py	Aucun test	CRUD non vérifié

Module	Raison	Risque
web/create_admin.py	CLI bootstrap script	Échec silencieux au démarrage
web/business_api.py	Exercé indirectement via TestClient	Pas d'isolation ciblée
web/app_auth_helpers.py	Importé seulement, pas de test	current_user_or_401 manqué

Tableau 5 — Modules backend sans test dédié.

6.3 Couverture frontend — composants sans test

Au-delà de Home/LabBench/SuperAdmin déjà cités (F-HIGH-26), la liste des composants sans test est longue : DashboardView, MissionPlanner, PreFlightChecklist, TelemetryCharts, FlightHistory, MaintenanceView, FleetRadar, FleetMap, DigitalTwinExplorer, MechanicalTwinEditor, EngineeringCadWorkbench, DesignStudioView, VirtualAssemblyWorkshop, VirtualSimulationCenter, VirtualFlightTestHall, HumanCommandCenter, AeronauticalCenterView, MissionOperationsView, AeroLabPanel, AIChatBox, MavlinkTelemetryPanel, LocalMfaLogin, LabBenchHardwareConfig, ValidationAuditJournal, etc. Les flux utilisateur critiques sans E2E : (1) login OIDC → création projet → save → reload (P1-03 depuis le 14 octobre, toujours ouvert) ; (2) library → drawing → LABBENCH step-up → trial → measurement persistence → reload ; (3) isolation 2-user (test backend explicitement skippé) ; (4) F5 restore workspace ; (5) BroadcastChannel multi-tab ; (6) callback Entra réel.

6.4 Qualité des tests — flakiness, skips, hardcoded

Trois patterns posent problème : (1) `test_functional_regressions.py:183` est marqué `@pytest.mark.skip` avec la raison « TestClient cookie/header isolation for User B requires a real browser or a separate ASGI transport » — **l'invariant d'isolation 2-user n'est donc pas vérifié**. (2) `test_v6_real_postgresql.py` importe `pgserver` qui **n'est pas dans requirements.txt** — les 4 tests ERROR en CI standard. (3) `test_labbench_api.py:10` utilise une base SQLite partagée hardcodée `sqlite:////tmp/aeromind-turbo-tests.db` sans PID — des runs successifs peuvent polluer l'état.

Credentials hardcoded acceptables (test-only) : `test_admin_account_registry.py:56,93` avec `"Une phrase de passe unique 2026!"`. En revanche, `test_demo_email_config.py:8-9` valide explicitement que `default_demo_email="mcgregoralvin14@gmail.com"` est accepté — ce qui contredit directement la note du README § « Premier administrateur ».

6.5 Intégration CI — .github/workflows/ci.yml

Le workflow CI comporte 5 jobs parallèles à chaque pull_request et push sur main/master : (1) **backend** — matrix Python 3.11/3.12, service postgres:16, pip install + pytest + pip-audit + psql migrations ; (2) **frontend** — Node 22, pnpm 10.18.0, pnpm audit + check + test +

build ; (3) **docker** — docker build + Trivy scan + readiness check ; (4) **sast** — CodeQL Python + JS/TS ; (5) **i18n** — `pnpm i18n:gate`. Les jobs pytest, vitest, tsc, vite build et i18n gate sont bloquants. Mais **les trois scanners sécurité ne le sont pas** — voir D-CRIT-09.

Le paramètre `--maxfail=5` est trop agressif pour une suite de 220 tests : une fois 5 erreurs accumulées (par exemple 4 ERROR de `pgserver` + 1 échec unrelated), CI s'arrête sans exécuter le reste. `pytest-asyncio` est installé mais aucun `asyncio_mode` n'est déclaré dans `pytest.ini` ou `pyproject.toml` — warnings de collection probables. Aucun seuil de couverture (`--cov=web --cov-fail-under=...` n'est configuré). Aucun job Playwright.

6.6 Chronologie des audits et livraisons

L'archive contient 75 rapports d'audit, de livraison et de modernisation accumulés depuis le 13 octobre 2026. Le tableau ci-dessous résume les jalons principaux :

Date	Type	Fichier	Thème
13 oct	Diagnostic	RAPPORT_DIAGNOSTIC_AEROMIND_V2_1.md	Phase 0 — sync ruptures
14 oct	Audit	RAPPORT_AUDIT_TECHNIQUE_GLOBAL_AEROMIND_2026-10-14.md	AUD-CRIT-01..05 backend non démarrable
14 oct	Audit	AUDIT_TECHNIQUE_AEROMIND_2026-10-14.md	P1-01..04, P2-01..06, P3 — gates A-F
15 oct	Livraison	RAPPORT_MIGRATION_MAVLINK_2026-10-15.md	mavlink_observer read-only
16 oct	Audit	AUDIT_I18N_PHASE0_V2_2026-10-16.md	A-01..A-12 i18n — REJETÉE
16 oct	Clôture	RAPPORT_CLOTURE_I18N_2026-10-16.md	NON CONFORME — 8 137 hardcoded
18 oct	Plan	PLAN-MODERNISATION-AEROMIND-2026-10-18.md	Roadmap tranches 1-6
22 oct	Modernis.	MODERNISATION_LABBENCH_V2..V5_2026-10-22.md	v2-v5 LABBENCH
24 oct	Final V3.1	RAPPORT_FINAL_V3_1_2026-10-24.md	6-gate hardware auth — 197 passed
25 oct	Conformité	RAPPORT_CONFORMITE_V5_2026-10-25.md	R01-R20 (14 PASS, 6 BLOCKED)
26 oct	Livraison	LIVRAISON-CORRECTION-IDENTITE-AEROMIND-LABBENCH.md	Re-instate mcgregoralvin14@gmail.com (!)
28 oct	Livraison	LIVRAISON-CORRECTION-CHARGEMENT-SYNCHRONISATION-2026-10-28.md	Réservation SEC-08 step-up
29 oct	Livraison	LIVRAISON-MODERNISATION-AUTH-NAVSAT-LABBENCH-2026-10-29.md	Pas de pytest exécuté

Tableau 6 — Chronologie résumée des audits et livraisons (13 oct → 29 oct 2026).

6.7 Défauts récurrents — identifiés à plusieurs reprises sans correction

L'analyse des 75 rapports révèle des défauts qui apparaissent dans plusieurs audits successifs sans être corrigés — signal d'alerte sur la fiabilité des déclarations de closure :

Défaut récurrent	Première occurence	Dernière	Compte
OIDC non configuré en env d'audit	14 oct	26 oct	⩾ 6
Pas de suite E2E navigateur (Playwright)	13 oct	25 oct (R18 BLOCKED)	⩾ 8
Migration PostgreSQL live jamais exécutée	14 oct	25 oct (R09-R11 BLOCKED)	⩾ 5
Home.tsx monolithe — pas d'éclatement	14 oct (12 956 lignes)	29 oct (14 315 lignes)	⩾ 4
Bundle size > 500 KiB	14 oct	29 oct	⩾ 10
SITL réel jamais testé (pas d'ArduPilot)	15 oct	25 oct (R19 BLOCKED)	⩾ 4
i18n 8 137 chaînes hardcoded résiduelles	16 oct	16 oct (NON CONFORME)	⩾ 3
mcgregoralvin14@gmail.com hardcoded super-admin	14 oct	26 oct (RE-INSTATED)	⩾ 4
Pas de CI step pour browser SITL/HIL	14 oct	25 oct	⩾ 5

Tableau 7 — Défauts récurrents identifiés sans correction effective.

6.8 Statut réel SEC-01 à SEC-08 — README vs audit vs code

Le README § « Failles corrigées (Lots A-F) » déclare SEC-01 à SEC-07 « ✓ Corrigé » à l'exception de SEC-06 marqué « ◐ Partiel ». L'inspection du code réel donne un tableau plus nuancé :

ID	README claim	Code reality	Statut réel
SEC-01	✓ Corrigé (Gate 0 context)	authorize_physical_command exige project_id/drone_id/actor_id	FIXED
SEC-02	✓ Corrigé (owner_id binding)	MavlinkPool.register exige owner_id, project_id	FIXED (pool layer)
SEC-03	✓ Corrigé (HMAC + atomic consume)	B-CRIT-02: _hmac_secret = "insecure-dev-only"	NOT FIXED (security theater)
SEC-04	✓ Corrigé (DB-backed + fail-closed)	GeofenceRecord DB-backed + assert fail-closed	FIXED
SEC-05	✓ Corrigé (email supprimé)	mcgregoralvin14@gmail.com toujours dans config.py:55,59, admin.py:65	NOT FIXED

ID	README claim	Code reality	Statut réel
SEC-06	◐ Partiel (frontend cleanup TODO)	Backend workspace header OK, frontend cleanup partiel	PARTIAL
SEC-07	√ Corrigé (auth + consent + state)	mavlink_telemetry_api auth + consent	FIXED
SEC-08 (new)	Non listé dans README	SEC-08: /api/lab/sessions/step-up accepte payload.roles	NOT FIXED (CRITICAL)

Tableau 8 — Statut réel SEC-01 à SEC-08.

CRITIQUE T-HIGH-27 — SEC-08 privilège escalation /api/lab/sessions/step-up

Le endpoint `POST /api/lab/sessions/step-up` (`web/labbench_canonical_api.py:179-204`) accepte `roles=set(payload.roles)` depuis le corps de la requête, après simple validation du nom du rôle contre `ROLES`. **Un utilisateur peut s'auto-assigner n'importe quel rôle** (admin, approver, operator) en l'incluant dans la requête.

Le rapport `LIVRAISON-CORRECTION-CHARGEMENT-SYNCHRONISATION-2026-10-28.md` le reconnaît explicitement : « les rôles sont acceptés depuis payload.roles après validation de leur nom et transmis à la session Step-up. Ce flux doit être corrigé avant toute exposition en production. Ne pas considérer cette réserve comme résolue. »

Correctif — calculer les rôles server-side depuis AdminAccountRecord

```
--- web/labbench_canonical_api.py:179-204 (actuel)
-roles = set(payload.roles) # client-supplied, juste validés contre ROLES
+++ web/labbench_canonical_api.py:179-204 (corrigé)
+from .admin import account_for_identity
+acct = account_for_identity(user['email'], user.get('oidc_claims', {}))
+if not acct:
+ raise HTTPException(403, detail={
+ 'code': 'FORBIDDEN',
+ 'message': 'Compte non pré-enregistré. Contactez un super-admin.'
+ })
+# Rôles derivés du compte server-side, jamais trustés du client.
+roles = {acct.role}
+# Demande de downgrade seulement (least-privilege).
+if payload.roles:
+ requested = set(payload.roles) & ROLES
+ roles = roles & requested # intersect — ne peut que restreindre
```

6.9 Contradictions entre audits successifs

L'inspection recense **15 contradictions majeures** entre les audits et livraisons successives. Les plus préoccupantes :

ID	Contradiction	Source A	Source B
C-1	SEC-05 « fixed » vs email still hardcoded	README L182-184	config.py:55,59 ; admin.py:65

ID	Contradiction	Source A	Source B
C-2	validation/*.log cités comme preuves mais répertoire vide	RAPPORT_FINAL_V3_1 § 2.5	ls -la validation/ → empty
C-3	Count tests 197 → 232 → 239 → « 281 »	V3.1 (24 oct)	LIVRAISON 29 oct
C-4	« All reception clauses PASS » vs STEP-UP MFA broken	RAPPORT_FINAL_V3_1 § 8	LIVRAISON 28 oct
C-5	« HARDWARE_CONTROL seul insuffisant » vs step-up accepts roles	V3.1 § 2.1	Code reality
C-6	i18n « NON CONFORME » vs « 2617 leaf keys synchronisés »	Clôture 16 oct	V3.1 § 1
C-11	Step-up « TOTP 30s » vs code uses daily code	MANUEL-OPERATEUR § 2	labbench_canonical_api.py :186-191
C-12	« 100 % contract » vs CRITICAL security reservation	LIVRAISON 26 oct	LIVRAISON 28 oct
C-14	__manus__ leftovers supprimés vs toujours présents	MODERNISATION_LABBENCH_V2 § 61	vite.config.ts:40 ; public/__manus__/

Tableau 9 — Contradictions majeures entre audits successifs (extrait).

6.10 Lots A-F — statut réel

Les six lots du cahier des charges (A-F) sont globalement traités, avec deux lots partiels :

Lot	Périmètre	Statut	Note
A	Sécurité matérielle & approbation indépendante	✓ Traité (11 tests)	sign_hardware_review, HMAC, atomic consume
B	PostgreSQL, migrations, persistance	◐ PARTIAL	R09/R10/R11 BLOCKED — PG live jamais exécuté
C	Sync canonique Bibliothèque/Dessin/LABBENCH	✓ Traité (5 tests)	DroneRevision immutable, active_revision_id
D	LABBENCH identité, sessions, CSRF, isolation	✓ Traité (15 tests)	R03 TOTP/CSRF, R04 session DB, R08 2-user (HTTP only)
E	Sciences des essais, missions, télémétrie, SITL	◐ PARTIAL	R19 (real SITL) BLOCKED
F	Qualité frontend, observabilité, CI	✓ Traité	validation/*.log cités mais répertoire vide (C-2)

Tableau 10 — Statut réel des lots A-F.

CHAPITRE 7 · DEVOPS & DÉPLOIEMENT

Inspection DevOps & Déploiement

La chaîne DevOps d'AeroMind est structurée : Dockerfile multi-stage, docker-compose, render.yaml Blueprint, CI GitHub Actions 5 jobs, migrations SQL datées avec rollback documenté. Cependant, l'inspection révèle **18 défauts actionnables** dont **4 critiques** qui rendent le déploiement production non défendable. Les trois préoccupations principales : (1) la sécurité de la chaîne CI est entièrement silenciée (scanners non bloquants) ; (2) le Blueprint Render embarque une adresse Gmail personnelle en super-admin par défaut ; (3) il n'existe pas d'outil de migration digne de ce nom (pas d'Alembic, pas de table schema_migrations, migrations TiDB non idempotentes).

7.1 Dockerfile — multi-stage, mais améliorations nécessaires

Le Dockerfile est globalement bien structuré : multi-stage (stage `frontend-build` avec `node:22-bookworm-slim` → stage `runtime` avec `python:3.12-slim`), création d'un utilisateur non-root `aeromind` via `addgroup` + `adduser --system`, `USER aeromind` avant `EXPOSE/CMD`, `requirements.txt` copié et pip installé avant le code applicatif (réutilisation du cache de couches), `PIP_NO_CACHE_DIR=1`, `PYTHONDONTWRITEBYTECODE=1`, `PYTHONUNBUFFERED=1`. Aucun `ARG` avec secret, aucun `apt install` (slim suffit).

Trois améliorations sprint 2 : (1) **CMD shell form** — `CMD ["sh", "-c", "uvicorn ... ${PORT:-10000}"]` exécute uvicorn comme enfant de `/bin/sh` PID 1 ; SIGTERM de Render prend ~10s à se propager, ce qui peut causer des shutdowns non propres (requêtes en vol droppées, connexions DB non fermées). Préférer la forme exec ou ajouter `--timeout-graceful-shutdown 5`. (2) **Pas de HEALTHCHECK directive** dans l'image — Render utilise `healthCheckPath` externe, mais tout autre environnement (docker local, k8s, fly.io) n'aura pas de health status. (3) **.dockerignore incomplet** — 15 lignes seulement, n'exclut pas `docs/`, `engineering/`, `tests/`, `*.md` (50+ rapports), `AUDIT_I18N_*.json` (~2.5 MB), `.github/`, ce qui alourdit le contexte de build et risque de faire fuiter des fichiers dans les layers.

Correctif — Dockerfile optimisé

```
# Dockerfile (extrait corrigé)
+HEALTHCHECK --interval=30s --timeout=5s --start-period=20s --retries=3 \
+ CMD python -c "import urllib.request,sys; sys.exit(0 if
urllib.request.urlopen('http://127.0.0.1:${PORT:-10000}/api/ready', timeout=3).status==200 else 1)"

-CMD ["sh", "-c", "uvicorn web.app:app --host 0.0.0.0 --port ${PORT:-10000}"]
+CMD ["uvicorn", "web.app:app", "--host", "0.0.0.0", "--port", "10000", \
+ "--workers", "2", "--limit-concurrency", "100", \
+ "--timeout-keep-alive", "30", "--timeout-graceful-shutdown", "5"]

# .dockerignore (extrait complété)
```

```
+docs
+tests
+engineering
+*.md
+AUDIT_*.json
+AUDIT_*.txt
+.github
+frontend/client/public/assets/aeromind-cell-library/*.obj
+frontend/client/public/assets/aeromind-cell-library/*.stl
+*.scad *.png *.webp
+scripts/hil
+scripts/sitl
```

7.2 docker-compose.yml — secret hardcoded, pas de networks

HIGH D-HIGH-12 — POSTGRES_PASSWORD hardcoded dans le repo

Le fichier `docker-compose.yml:8` déclare `POSTGRES_PASSWORD: aeromind_dev_only`. Bien que la valeur soit marquée « dev_only », tout développeur pointant le même compose sur un Postgres staging sans override partagerait ce mot de passe. Aucun réseau déclaré (bridge par défaut, pas d'isolation entre services). Aucun `mem_limit`/`cpus` — un uvicorn runaway peut OOM l'hôte. Aucun `init: true` — PID 1 est `sh` (cf. CMD shell form) qui ne forward pas SIGTERM à uvicorn. Aucune rotation de logs (`json-file` par défaut, sans `max-size`/`max-file`).

Correctif — docker-compose durci

```
# docker-compose.yml (extrait corrigé)
services:
db:
image: postgres:16-alpine
- POSTGRES_PASSWORD: aeromind_dev_only
+ POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?POSTGRES_PASSWORD is required}
volumes: [aeromind-postgres:/var/lib/postgresql/data]
+ networks: [aeromind-net]
+ logging: { driver: json-file, options: { max-size: "10m", max-file: "3" } }
app:
build: .
+ init: true # PID 1 = tini, forward SIGTERM proprement
+ networks: [aeromind-net]
+ mem_limit: 1g
+ cpus: "1.5"
+ logging: { driver: json-file, options: { max-size: "10m", max-file: "3" } }
+networks:
+ aeromind-net:
+ driver: bridge
```

7.3 render.yaml — Blueprint Render

CRITIQUE D-CRIT-08 — mcgregoralvin14@gmail.com en super-admin render.yaml

Le Blueprint Render `render.yaml:49-54` hardcode :

```
AEROMIND_SUPERADMIN_EMAILS: "mcgregoralvin14@gmail.com"
AEROMIND_TRIAL_EMAILS: "mcgregoralvin14@gmail.com"
DEFAULT_DEMO_EMAIL: "mcgregoralvin14@gmail.com"
```

Quiconque applique ce Blueprint verbatim accorde les droits super-admin à une adresse Gmail personnelle au premier déploiement. Combiné avec `autoDeploy: true` (ligne 29), c'est un footgun d'escalade de privilèges. La doc `DEPLOYMENT_RENDER_ENVIRONMENT.md` prévient explicitement contre cette pratique — mais le Blueprint fait exactement ça.

HIGH D-HIGH-13 — Free tier Render web + DB (DB supprimée après 90 jours idle)

Le `render.yaml:24` déclare `plan: free` pour le web service, et `render.yaml:97` déclare `plan: free` pour PostgreSQL. Conséquences : (1) le web service Render free **s'endors après 15 min d'inactivité** (cold starts 30-60s) ; (2) **Render free PostgreSQL est supprimé après 90 jours d'inactivité** avec toutes les données — politique Render documentée. Pour un cockpit d'ingénierie drone avec logs d'audit append-only, perdre la DB après 90 jours idle est inacceptable. Le free tier a aussi 512 MB RAM et 0.1 CPU — insuffisant pour uvicorn + SQLAlchemy + Vite static serving.

HIGH D-HIGH-14 — autoDeploy: true sur main sans status checks requis

Le `render.yaml:29` déclare `autoDeploy: true`. Chaque push sur main déclenche un déploiement production immédiat. Combiné avec les scanners CI non bloquants (D-CRIT-09), un commit vulnérable sur main est déployé en production sans revue humaine. Aucune branche protégée n'exige que les 5 jobs CI passent.

Correctif — render.yaml durci

```
# render.yaml (extrait corrigé)
services:
- type: web
name: aeromind
env: docker
region: frankfurt
- plan: free
+ plan: starter # $7/mo — pas de sleep, 512 MB RAM → 1 GB
healthCheckPath: /api/ready
- autoDeploy: true
+ autoDeploy: false # déploiements manuels via Render Dashboard après CI vert
envVars:
- - key: AEROMIND_SUPERADMIN_EMAILS
- value: "mcgregoralvin14@gmail.com"
+ - key: AEROMIND_SUPERADMIN_EMAILS
+ sync: false # valeur saisie manuellement au premier déploiement
# ... idem pour TRIAL_EMAILS, DEFAULT_DEMO_EMAIL
- - key: AEROMIND_AUTO_CREATE_SCHEMA
- value: "true"
+ - key: AEROMIND_AUTO_CREATE_SCHEMA
+ value: "false"
```

```
+ preDeployCommand: "bash scripts/apply_migrations.sh $DATABASE_URL"

databases:
- name: aeromind-db
region: frankfurt
- plan: free
+ plan: starter # $7/mo — backups automatiques quotidiens, 7 jours de rétention
```

7.4 .env.example — variables critiques manquantes

L'inventaire complet recense 27 variables documentées dans `.env.example`, mais **8 variables critiques utilisées par le code en sont absentes** : `AEROMIND_SITL_ACTIVE`, `AEROMIND_HIL_ACTIVE`, `AEROMIND_AUTO_CREATE_SCHEMA`, `AEROMIND_WORKSPACE_TTL_SECONDS`, `AEROMIND_MISSION_REQUEST_TIMEOUT_S`, `AEROMIND_MISSION_ACK_TIMEOUT_S`, `AEROMIND_DB_CA`, `AEROMIND_MAVLINK_OBSERVER_URL`. À cela s'ajoutent les variables d'observabilité absentes : `SENTRY_DSN`, `LOG_LEVEL`, `AEROMIND_DB_POOL_SIZE`, `AEROMIND_DB_MAX_OVERFLOW`, `AEROMIND_RATE_LIMIT_PER_MIN`.

Le placeholder `AEROMIND_SESSION_SECRET=remplacer-par-un-secret-aleatoire-d-au-moins-32-octets` fait 52 caractères et **passe le validateur** (`len >= 32`). Un opérateur distrait pourrait déployer avec ce placeholder comme secret de session production — toutes les sessions seraient signées avec une clé publiquement connue. Correctif : placeholder vide + check au démarrage qui refuse de booter si `AEROMIND_ENV=production` et que le secret égal le placeholder.

7.5 CI/CD workflows — scanners sécurité silenciés

> **CRITIQUE D-CRIT-09 — Tous les scanners CI silenciés par || true**
>
> Le workflow `.github/workflows/ci.yml` exécute bien :
>
> Ligne 42 : `pip-audit -r requirements.txt --ignore-vuln GHSA-XXXX || true`
> Ligne 82 : `pnpm audit --audit-level=high --prod || true`
> Ligne 108 : Trivy avec `exit-code: "0"`
>
> Conséquence : **une CVE critique RCE ne bloquerait pas la fusion d'une PR**. Le paramètre `--ignore-vuln GHSA-XXXX` est un placeholder non remplacé — si pip-audit passe en mode strict, le build casse pour la mauvaise raison. Aucune branche protégée n'exige que les 5 jobs CI passent.

Correctif — CI durcie

```
# .github/workflows/ci.yml (extrait corrigé)

# Job backend
- run: pip-audit -r requirements.txt --ignore-vuln GHSA-XXXX || true
+ run: pip-audit -r requirements.txt # Pas de || true. Pas de --ignore-vuln GHSA-XXXX.
+ # Pour ignorer un advisory réel, documenter:
+ # --ignore-vuln GHSA-1234-5678-9abc # false positive: dep X sur Python 3.11 only
```

```
# Job frontend
- run: pnpm audit --audit-level=high --prod || true
+ run: pnpm audit --audit-level=high --prod # Pas de || true

# Job docker
- uses: aquasecurity/trivy-action@master
- with:
- exit-code: "0"
+ uses: aquasecurity/trivy-action@0.20.0 # Pin to specific tag, not @master
+ with:
+ exit-code: "1" # Fail build on HIGH/CRITICAL CVEs
+ severity: "HIGH,CRITICAL"

# Top-level: concurrency pour cancel redundant runs
+concurrency:
+ group: ci-${{ github.ref }}
+ cancel-in-progress: true
```

Quatre autres problèmes CI à corriger : (1) **Actions non SHA-pinnés** — `pnpm/action-setup@v4`, `actions/setup-node@v4`, `aquasecurity/trivy-action@master` sont des tags mutables. Si `trivy-action@master` est compromis (vol de token mainteneur), une action malicieuse tourne en CI avec scope `GITHUB_TOKEN` — backdoor possible dans l'image Docker. **@master est le plus dangereux** (pas de garantie d'immutabilité). Pinner à un SHA ou à un tag versionné. (2) **Pas de permissions: block** sur 4 des 5 jobs — défaut `contents: write` (legacy) ou autre, principe de moindre privilège violé. (3) **Job i18n dupliqué** — le job `i18n` de `ci.yml` et le workflow `i18n-gate.yml` exécutent la même commande `pnpm i18n:gate` sur les mêmes triggers — minutes CI doublées. (4) **Pas de timeout-minutes** — un job bloqué tourne les 6h par défaut.

Le `ci.yml:47-52` applique les migrations PostgreSQL avec `|| true` sur les deux premières : `psql -f 2026-10-21_admin_account_registry.sql || true` et `psql -f 2026-10-21_validation_audit_events.sql || true`. Ceci masque toute erreur, y compris « table users does not exist » (cible FK). La troisième migration `2026-10-25_canonical_labbench_schema.sql` n'a pas `|| true` — elle échouerait si les tables `drones`/`missions` n'existent pas (le schema de base n'est créé que par `create_all`, pas par une migration). **CI ne valide pas réellement que les migrations s'appliquent proprement à un PostgreSQL frais.**

7.6 Migrations de base de données

CRITIQUE D-CRIT-10 — Migration TiDB non idempotente + tables manquantes

> Le fichier `migrations/2026-10-25_canonical_labbench_schema_tidb.sql:189-192` utilise :
>
> ```
> ALTER TABLE missions ADD COLUMN upload_state VARCHAR(20) DEFAULT 'pending';
> ALTER TABLE missions ADD COLUMN upload_started_at DATETIME NULL;
> ALTER TABLE missions ADD COLUMN upload_error TEXT NULL;
> ```
>
> sans `IF NOT EXISTS` (TiDB ne supporte pas cette syntaxe). Le commentaire à la ligne 189 le reconnaît : « idempotent — TiDB does not support IF NOT EXISTS on ADD COLUMN » — mais la migration n'est PAS idempotente. La ré-exécuter échoue avec `Duplicate column name`.
>
> Plus grave : la variante TiDB crée 8 tables contre 11 en PostgreSQL. Manquent : `anonymous_workspaces` (V5 LOT B / R-04, référencé dans `web/app.py:230-329`), `gate_validations` (G0-G5 gate tracking). L'application TiDB aurait des erreurs « Table 'anonymous_workspaces' doesn't exist ».

Correctif — adopter Alembic + préDeployCommand

```
# Étape 1: Initialiser Alembic
$ cd /home/z/my-project/aeromind_inspection/aeromind/
$ alembic init migrations
$ # Configurer alembic.ini pour utiliser DATABASE_URL
$ # Configurer migrations/env.py pour import Base.metadata

# Étape 2: Générer la migration initiale depuis Base.metadata
$ alembic revision --autogenerate -m "initial schema"

# Étape 3: Convertir les migrations SQL existantes en migrations Alembic
$ # 2026-10-21_admin_account_registry.sql → revision 0001
$ # 2026-10-21_validation_audit_events.sql → revision 0002
$ # 2026-10-25_canonical_labbench_schema.sql → revision 0003

# Étape 4: preDeployCommand dans render.yaml
# render.yaml:
# preDeployCommand: "alembic upgrade head"

# Étape 5: Idempotence TiDB — Alembic gère automatically via op.invoke()
```

Aucune table `schema_migrations` ne track quelles migrations ont été appliquées. L'ordre d'application est implicite (alphabétique : `2026-10-21_*` puis `2026-10-25_*`). Ré-déployer peut tenter de ré-appliquer une migration déjà appliquée — échec TiDB, no-op silencieux PostgreSQL. Aucun rollback exécutable : les scripts de rollback (commentés en bas des fichiers SQL) doivent être copiés-collés manuellement dans `psql`. La doc `docs/PLAN-ROLLBACK-LABBENCH-01-FR.md:39-44` référence ce rollback mais fournit un SQL différent (drop 2 tables au lieu de 11).

Il manque aussi un **bootstrap PostgreSQL schema migration** : la migration canonique déclare des FK vers `users(id)`, `projects(id)`, `drones(id)`, `missions(id)` et fait des `ALTER TABLE drones` et `missions` — mais aucune de ces tables n'est créée par une migration. Elles existent seulement dans `Base.metadata` (déclarées dans `web/models.py`). En production avec `AUTO_CREATE_SCHEMA=false` (recommandé), il n'y a **aucun moyen d'initialiser un PostgreSQL frais** sans laisser l'app `create_all` une fois (ce qui défait le but de

AUTO_CREATE_SCHEMA=false). Correctif sprint 2 : générer un migrations/0001_base_schema.sql via pg_dump --schema-only après un create_all one-shot.

7.7 Backup & disaster recovery

Render free PostgreSQL n'inclut pas de backup automatique (seuls les plans standard et supérieur ont backup quotidien 7 jours de rétention). Le plan de rollback docs/PLAN-ROLLBACK-LABBENCH-01-FR.md:18 référence backups/pre-migration-YYYY-MM-DD.sql.gz (manuel), mais aucun script n'automatise ceci. Le répertoire backups/ n'existe pas dans le repo. La doc docs/PLAN-ROLLBACK-LABBENCH-01-FR.md:154 référence docs/PROCEDURE-INCIDENT-FR.md « à compléter » — le runbook d'incident n'existe pas. La politique de rétention 90 jours DB / 1 an audit_logs est documentée mais **non enforceée**.

Bonnes pratiques relevées : le plan rollback par lot (docs/PLAN-ROLLBACK-LABBENCH-01-FR.md:24-105) est documenté pour chaque lot A-F avec fichiers affectés, SQL de rollback, commande git revert et effet. La procédure d'urgence §4 séquence correctement : HARDWARE_CONTROL=false → revoke sessions → verify /api/health → git revert → restore DB. Les sentinelles post-rollback (T01, T08, T15) sont documentées. Les critères « no-rollback » (G5 physical trials done, signed recette, 30 jours en G5) sont clairs.

7.8 Concerns opérationnels — observabilité, rate limiting, CSP

> **HIGH D-HIGH-18 — Logging print() only, pas de Sentry, pas de /metrics, pas de rate limit**
>
> Toute la logging applicative passe par print(..., flush=True) (11 occurrences dans web/app.py, plus dans web/admin.py, etc.). Seuls 3 modules utilisent logging.getLogger. Pas de logging structuré (JSON), pas de LOG_LEVEL, pas de correlation ID. **Aucun Sentry** — aucune intégration sentry_sdk importée nulle part dans web/. Aucun endpoint /metrics Prometheus. Aucun rate limiting — les endpoints /api/auth/dev-login, /api/auth/oidc/login, /api/auth/oidc/callback peuvent être brute-forcés.

Correctif — ajouter Sentry + structlog + slowapi + CSP

```
# requirements.txt (ajouts)
+sentry-sdk[fastapi]==2.18.0
+structlog==24.4.0
+slowapi==0.1.9

# web/app.py (lifespan, ajouts)
+import sentry_sdk
+from sentry_sdk.integrations.fastapi import FastApiIntegration
+sentry_sdk.init(
+ dsn=os.getenv("SENTRY_DSN"),
+ environment=settings.environment,
```

```
+ traces_sample_rate=0.1,
+ integrations=[FastApiIntegration()],
+)

# web/app.py (rate limiting sur endpoints auth)
+from slowapi import Limiter
+from slowapi.util import get_remote_address
+limiter = Limiter(key_func=get_remote_address)
+app.state.limiter = limiter
+
+@app.post("/api/auth/dev-login")
+@limiter.limit("5/minute")
+async def dev_login(request: Request, ...): ...

# web/app.py (CSP — voir F-CRIT-12 correctif pour le code complet)
```

Bonnes pratiques DevOps à conserver : (1) **HSTS header** en production (`max-age=31536000; includeSubDomains`) — manque `preload` mais c'est mineur ; (2) **TrustedHostMiddleware** avec `settings.hosts` ; (3) **TLS terminé par Render** (pas de gestion de certificat) ; (4) cookie `Secure` en production ; (5) `pg_isready` avec 20 retries × 5s ; (6) endpoints `/api/health` et `/api/ready` implémentés correctement (503 si DB unreachable) ; (7) lockfile committé et `--frozen-lockfile` en CI et Docker ; (8) versions Python et Node alignées entre CI et Docker (Python 3.12, Node 22) ; (9) PostgreSQL 16 aligné entre CI, docker-compose et Render.

CHAPITRE 8 · PLAN DE REMÉDIATION P0

Plan de remédiation P0 — Bloquants production

Ce chapitre consolide les douze correctifs CRITICAL à appliquer **avant tout déploiement production**. Chaque correctif est accompagné de l'emplacement précis, de l'impact et d'un snippet de code ou d'un diff unifié directement applicable. L'effort total est estimé à **3 à 5 jours-homme** de travail concentré. Aucun autre travail fonctionnel ne doit être engagé tant que la branche `sprint-1-critical-fixes` n'a pas passé la CI durcie et que la suite pytest + vitest est verte.

P0-01 — Définir current_user_or_401 (B-CRIT-01)

Fichier : `web/app_auth_helpers.py` (25 lignes actuellement).
Impact : débloque l'ensemble du routeur canonique `/api/lab/*` (12 endpoints, 737 lignes).
Effort : 5 minutes (4 lignes à ajouter).

```
# web/app_auth_helpers.py — ajouter à la fin du fichier

def current_user_or_401(request) -> dict:
"""Return the authenticated user or raise 401 immediately."""
user = current_user(
request.cookies.get('aeromind_session'),
request.headers.get('authorization'),
)
if user is None:
from fastapi import HTTPException
raise HTTPException(
status_code=401,
detail={'code': 'UNAUTHENTICATED', 'message': 'Authentication required.'}
)
return user
```

Test de non-régression : ajouter `tests/test_app_auth_helpers.py` qui vérifie que l'import fonctionne, que la fonction retourne l'utilisateur si authentifié, et qu'elle lève `HTTPException(401)` sinon.

P0-02 — Fixer _hmac_secret (B-CRIT-02)

Fichier : `web/labbench_auth.py:381-383`.
Impact : restaure l'intégrité des tokens matériels HMAC. Sans ce fix, tout attaquant qui lit une ligne `hardware_tokens` (dump DB, injection SQL, fuite log) peut forger un HMAC valide pour n'importe quel `(token_id, project_id, drone_id, command, expires_at)`.
Effort : 10 minutes.

```
# web/labbench_auth.py:381-383 — remplacer

def _hmac_secret() -> bytes:
"""Return the HMAC secret for hardware token signing.

Pydantic Settings expose l'attribut Python, pas l'alias env.
getattr(settings, "AEROMIND_SESSION_SECRET") retourne toujours "".
"""
secret = get_settings().session_secret
if not secret or len(secret) < 32:
raise RuntimeError(
"AEROMIND_SESSION_SECRET must be set and >= 32 bytes "
"for hardware token HMAC signing. Refusing to boot."
)
return secret.encode("utf-8")[:64]
```

P0-03 — Passer owner_id et project_id (B-CRIT-04, B-CRIT-05, B-HIGH-01)

Fichiers : `web/app.py` lignes 1202, 1227, 1304, 1427.
Impact : débloque l'enregistrement de drones dans le pool, le unregister, le pilot pour command, et la pose de géofence. Sans ces fixes, le pool MAVLink est inutilisable et la géofence fail-closed bloque toute commande.
Effort : 20 minutes (4 modifications + tests).

```
# web/app.py:1202 — pool.register
-entry = pool.register(drone_id, drone_name, connection_url)
+entry = pool.register(
+ drone_id, drone_name, connection_url,
+ owner_id=user['id'],
+ project_id=project_id or '',
+)

# web/app.py:1227 — pool.unregister
-mavlink_pool_service.get_pool().unregister(drone_id)
+mavlink_pool_service.get_pool().unregister(drone_id, owner_id=user['id'])

# web/app.py:1304 — pool.pilot (pour /api/hardware/drones/{id}/command)
-pilot = pool.pilot(drone_id)
+pilot = pool.pilot(drone_id, owner_id=user['id'])

# web/app.py:1427 — geofence_service.set_geofence
-geofence_service.set_geofence(geofence)
+geofence_service.set_geofence(geofence, owner_id=user['id'])
```

P0-04 — Enforcer CSRF sur routes state-changeantes (B-CRIT-03)

Fichier : `web/app.py:150-171` (router registration).
Impact : ferme la vulnérabilité CSRF sur tous les endpoints POST/PUT/PATCH/DELETE. Le cookie `aeromind_session` étant `SameSite=None; Secure` en production, toute requête state-changeante est actuellement vulnérable.
Effort : 30 minutes (ajout de dependency + tests).

```
# web/app.py — modifier la section d'enregistrement des routeurs

from .labbench_auth import verify_csrf_double_submit
_csrf_dep = [Depends(verify_csrf_double_submit)]

# Appliquer dependencies=_csrf_dep sur tous les routeurs state-changeants:
app.include_router(business_router, dependencies=_csrf_dep)
app.include_router(hardware_router, dependencies=_csrf_dep)
app.include_router(labbench_canonical_router, dependencies=_csrf_dep)
app.include_router(mission_planner_router, dependencies=_csrf_dep)
app.include_router(local_auth_router, dependencies=_csrf_dep)
app.include_router(oidc_router, dependencies=_csrf_dep)
app.include_router(admin_router, dependencies=_csrf_dep)
app.include_router(command_center_router, dependencies=_csrf_dep)
app.include_router(preflight_router, dependencies=_csrf_dep)
app.include_router(maintenance_router, dependencies=_csrf_dep)

# GET-only routers (lecture) peuvent rester sans CSRF:
# - mavlink_telemetry_router (GET /api/telemetry/mavlink/status) —
# mais POST /api/telemetry/mavlink/consent et DELETE restent CSRF-protégés
# - dashboard_router (GET /api/dashboard)
```

P0-05 — Scoper dashboard par Project.user_id (B-HIGH-02)

Fichier : `web/dashboard.py:60-65, 113-126`.
Impact : ferme la fuite cross-tenant des alertes récentes et du compteur `activeAlerts`. Violation directe de SEC-02.
Effort : 15 minutes.

```
# web/dashboard.py:60-65 et 113-126 — ajouter le scope user

from sqlalchemy import select as _select

_user_projects_subq = _select(Project.id).where(Project.user_id == user['id'])

# Ligne 60-65 (compteur activeAlerts)
-active_alerts = db.scalar(_select(func.count(AlertEvent.id)).where(AlertEvent.resolved == False))
+active_alerts = db.scalar(
+ _select(func.count(AlertEvent.id))
+ .where(AlertEvent.resolved == False)
+ .where(AlertEvent.project_id.in_(_user_projects_subq))
+)

# Ligne 113-126 (recent_alerts)
-recent_alerts = db.scalars(_select(AlertEvent).order_by(...).limit(5)).all()
+recent_alerts = db.scalars(
+ _select(AlertEvent)
+ .where(AlertEvent.project_id.in_(_user_projects_subq))
+ .order_by(AlertEvent.created_at.desc())
+ .limit(5)
+).all()
```

P0-06 — Synchroniser migration TiDB (B-CRIT-06, D-CRIT-10)

Fichier : `migrations/2026-10-25_canonical_labbench_schema_tidb.sql`.
Impact : rend la migration TiDB idempotente et complète (ajoute `lab_sessions.data_json`, `anonymous_workspaces`, `gate_validations`, `missions.upload_protocol`, `missions.upload_idempotency_key`).
Effort : 1 heure (diff avec variante PostgreSQL + test sur TiDB locale).

```
# 1. lab_sessions.id : VARCHAR(36) → VARCHAR(128) (aligné avec PostgreSQL et ORM)
- id VARCHAR(36) NOT NULL PRIMARY KEY,
+ id VARCHAR(128) NOT NULL PRIMARY KEY,

# 2. lab_sessions.data_json : ajouter (manquant en TiDB)
+ data_json LONGTEXT NOT NULL,

# 3. anonymous_workspaces et gate_validations : ajouter les CREATE TABLE manquants
+CREATE TABLE IF NOT EXISTS anonymous_workspaces (
+ id VARCHAR(128) NOT NULL PRIMARY KEY,
+ workspace_key VARCHAR(128) NOT NULL,
+ user_id VARCHAR(64) NULL,
+ created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
+ expires_at DATETIME NOT NULL,
+ data_json LONGTEXT NULL,
+ INDEX ix_anon_workspaces_key (workspace_key),
+ INDEX ix_anon_workspaces_expires (expires_at)
+) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

+CREATE TABLE IF NOT EXISTS gate_validations (
+ id BIGINT NOT NULL AUTO_INCREMENT PRIMARY KEY,
+ project_id VARCHAR(64) NOT NULL,
+ gate_name VARCHAR(32) NOT NULL,
+ state VARCHAR(20) NOT NULL DEFAULT 'pending',
+ approved_by VARCHAR(64) NULL,
+ approved_at DATETIME NULL,
+ evidence TEXT NULL,
+ created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
+ UNIQUE KEY uq_gate_project (project_id, gate_name)
+) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

# 4. missions : ajouter upload_protocol, upload_idempotency_key manquants
+SET @c1 = (SELECT COUNT(*) FROM information_schema.columns
+ WHERE table_schema=DATABASE() AND table_name='missions'
+ AND column_name='upload_protocol');
+SET @sql1 = IF(@c1=0,
+ 'ALTER TABLE missions ADD COLUMN upload_protocol VARCHAR(20) NULL',
+ 'SELECT 1'); PREPARE s1 FROM @sql1; EXECUTE s1; DEALLOCATE PREPARE s1;
+# Idem pour upload_idempotency_key (VARCHAR(128) NULL, avec index)
```

P0-07 — Persister telemetry consent en DB (B-HIGH-04)

Fichier : `web/mavlink_telemetry_api.py:47`.
Impact : le consentement télémétrie survive aux restarts worker et soit partagé entre workers Render (sinon 403 aléatoire en multi-worker).
Effort : 1 heure.

```
# web/mavlink_telemetry_api.py:47 — remplacer le dict in-memory
```

```
# AVANT:
# _consent_state: dict[str, dict] = {}

# APRÈS:
def _consent_key(user_id: str, drone_id: str) -> str:
return f"consent:{user_id}:{drone_id}"

def grant_consent(user_id: str, drone_id: str, scopes: list[str]) -> None:
from .session_store import _db_get_lab_session, _db_put_lab_session
# Stocker dans lab_sessions.data_json["consent"][drone_id]
sess = _db_get_lab_session(user_id)
if not sess:
return
sess.data.setdefault('consent', {})[drone_id] = {
'scopes': scopes,
'granted_at': datetime.utcnow().isoformat(),
}
_db_put_lab_session(sess)

def get_consent(user_id: str, drone_id: str) -> dict | None:
sess = _db_get_lab_session(user_id)
if not sess:
return None
return sess.data.get('consent', {}).get(drone_id)

def revoke_consent(user_id: str, drone_id: str) -> None:
sess = _db_get_lab_session(user_id)
if sess and 'consent' in sess.data:
sess.data['consent'].pop(drone_id, None)
_db_put_lab_session(sess)
```

P0-08 — Vérifier gates G0-G5 (B-HIGH-06)

Fichier : `web/hardware_gate.py:319-427` (dans `issue_hardware_confirmation_token`).
Impact : enforce l'exigence C-001 du cahier des charges. Sans ce fix, un opérateur peut émettre une commande MAVLink avec seulement N3+review+pool-registered, sans que les 6 gates G0-G5 ne soient approuvées.
Effort : 30 minutes.

```
# web/hardware_gate.py — après le check drone_registered_in_pool

from .canonical_models import GateValidation
from sqlalchemy import select as _select

required_gates = ['G0_gel', 'G1_securite', 'G2_fiabilite',
'G3_banc', 'G4_qualification', 'G5_exploitation']

approved = {row.gate_name for row in db.scalars(
_select(GateValidation).where(
GateValidation.project_id == project_id,
GateValidation.state == 'approved',
)
)}

missing = set(required_gates) - approved
```

```
if missing:
raise HardwareCommandError(
412, 'GATE_NOT_READY',
f'Gates requises non approuvées pour ce projet: {sorted(missing)}. '
f'EXIGENCE C-001 du cahier des charges.'
)
```

P0-09 — AUTO_CREATE_SCHEMA=false + preDeployCommand (B-HIGH-09)

Fichier : `render.yaml:33-34` + nouveau script `scripts/apply_migrations.sh`.
Impact : en production, le schéma est géré par migrations explicites et non par `Base.metadata.create_all()`. Élimine le risque de skew silencieux ORM/DB.
Effort : 30 minutes.

```
# render.yaml:33-34 — modifier
- AEROMIND_AUTO_CREATE_SCHEMA: "true"
+ AEROMIND_AUTO_CREATE_SCHEMA: "false"
+preDeployCommand: "bash scripts/apply_migrations.sh $DATABASE_URL"

# scripts/apply_migrations.sh (nouveau, chmod +x)
#!/usr/bin/env bash
set -euo pipefail
DB_URL="${1:?DATABASE_URL required}"
echo "[migrations] Applying to $DB_URL"
psql "$DB_URL" -v ON_ERROR_STOP=1 \
-f migrations/2026-10-21_admin_account_registry.sql \
-f migrations/2026-10-21_validation_audit_events_pg.sql \
-f migrations/2026-10-25_canonical_labbench_schema.sql
echo "[migrations] Done."
```

P0-10 — Supprimer mcgregoralvin14@gmail.com (D-CRIT-08, B-SEC-05)

Fichiers : `web/config.py:55,59,62`, `web/admin.py:65`, `.env.example:23-25`, `render.yaml:49-54`, `tests/test_demo_email_config.py:8-9`.
Impact : élimine l'escalade de privilèges par défaut. Tout déploiement Render qui n'écrase pas `AEROMIND_SUPERADMIN_EMAILS` accorde actuellement les droits super_admin à cette adresse Gmail.
Effort : 20 minutes (5 modifications + update tests).

```
# web/config.py — lignes 55, 59, 62
-superadmin_emails: str = Field('mcgregoralvin14@gmail.com', alias='AEROMIND_SUPERADMIN_EMAILS', ...)
+superadmin_emails: str = Field('', alias='AEROMIND_SUPERADMIN_EMAILS', ...)

-trial_emails: str = Field('mcgregoralvin14@gmail.com', alias='AEROMIND_TRIAL_EMAILS', ...)
+trial_emails: str = Field('', alias='AEROMIND_TRIAL_EMAILS', ...)

-default_demo_email: str = Field('mcgregoralvin14@gmail.com', alias='DEFAULT_DEMO_EMAIL', ...)
+default_demo_email: str = Field('', alias='DEFAULT_DEMO_EMAIL', ...)

# web/admin.py:65 — preferred est utilisé seulement si dans l'allowlist; supprimer quand même
```

```
-preferred = "mcgregoralvin14@gmail.com"
+# Preferred n'a plus de sens maintenant ; supprimer ou dériver de superadmin_emails.
+preferred = "" # À documenter: ce champ n'a plus de valeur par défaut.

# .env.example:23-25 — remplacer
-AEROMIND_SUPERADMIN_EMAILS=mcgregoralvin14@gmail.com
-AEROMIND_TRIAL_EMAILS=mcgregoralvin14@gmail.com
-DEFAULT_DEMO_EMAIL=mcgregoralvin14@gmail.com
+AEROMIND_SUPERADMIN_EMAILS= # OBLIGATOIRE en production. Ex: admin@yourdomain.com
+AEROMIND_TRIAL_EMAILS=
+DEFAULT_DEMO_EMAIL=

# render.yaml:49-54 — passer en sync: false
- - key: AEROMIND_SUPERADMIN_EMAILS
- value: "mcgregoralvin14@gmail.com"
+ - key: AEROMIND_SUPERADMIN_EMAILS
+ sync: false # valeur saisie manuellement au premier déploiement

# tests/test_demo_email_config.py — mettre à jour pour tester le comportement vide
-def test_default_demo_email():
- assert default_demo_email == 'mcgregoralvin14@gmail.com'
+def test_default_demo_email_empty():
+ """SEC-05: default must be empty, not a hardcoded personal email."""
+ assert default_demo_email == ''
```

P0-11 — CI scanners bloquants (D-CRIT-09)

Fichier : `.github/workflows/ci.yml` lignes 42, 82, 108.
Impact : une CVE critique RCE ne bloque actuellement pas la fusion. Avec ce fix, les CVE HIGH/CRITICAL bloquent les merges.
Effort : 15 minutes.

```
# .github/workflows/ci.yml

# Ligne 42 (job backend) — retirer || true et GHSA-XXXX
- run: pip-audit -r requirements.txt --ignore-vuln GHSA-XXXX || true
+ run: pip-audit -r requirements.txt
+ # Pour ignorer un advisory réel, documenter explicitement:
+ # --ignore-vuln GHSA-1234-5678-9abc # false positive: dep X sur Python 3.11

# Ligne 82 (job frontend) — retirer || true
- run: pnpm audit --audit-level=high --prod || true
+ run: pnpm audit --audit-level=high --prod

# Ligne 104-108 (job docker) — pinner trivy-action et exit-code 1
- uses: aquasecurity/trivy-action@master
- with:
- image-ref: aeromind:ci
- exit-code: "0"
- severity: "HIGH,CRITICAL"
+ uses: aquasecurity/trivy-action@0.20.0
+ with:
+ image-ref: aeromind:ci
+ exit-code: "1"
+ severity: "HIGH,CRITICAL"
+ ignore-unfixed: true
```

```
# Top-level — concurrency pour cancel redundant runs
+concurrency:
+ group: ci-${{ github.ref }}
+ cancel-in-progress: true

# Pour chaque job — ajouter
+ timeout-minutes: 30
+ permissions:
+ contents: read
```

P0-12 — Step-up roles server-side (T-HIGH-27, SEC-08)

Fichier : `web/labbench_canonical_api.py:179-204`.
Impact : ferme la vulnérabilité d'escalade de privilèges. Sans ce fix, n'importe quel utilisateur authentifié peut s'auto-assigner le rôle `admin`, `approver` ou `operator` en l'incluant dans `payload.roles`.
Effort : 45 minutes.

```
# web/labbench_canonical_api.py:179-204 — remplacer la logique des rôles

# AVANT:
# roles = set(payload.roles) # client-supplied, juste validés contre ROLES

# APRÈS:
from .admin import account_for_identity

acct = account_for_identity(user['email'], user.get('oidc_claims', {}))
if not acct:
raise HTTPException(403, detail={
'code': 'FORBIDDEN',
'message': 'Compte Microsoft Entra préexistant requis. '
'Contactez un super-admin pour inscription.'
})

# Rôles dérivés du compte server-side, jamais trustés du client.
roles = {acct.role}

# Demande de downgrade (least-privilege) seulement.
if payload.roles:
requested = set(payload.roles) & ROLES
roles = roles & requested # intersect — ne peut que restreindre

# Test de non-régression à ajouter dans tests/test_labbench_reception.py:
# def test_step_up_cannot_self_assign_admin(client, auth_reader):
# """SEC-08: payload.roles=['admin'] ne doit pas élever un reader."""
# resp = client.post('/api/lab/sessions/step-up',
# json={'project_id': '...', 'mfa_code': '...',
# 'roles': ['admin']}, # tentative d'élévation
# headers=auth_reader)
# assert resp.status_code == 200
# # Vérifier que la session a seulement le rôle 'reader', pas 'admin'
# sess_data = resp.json()
# assert 'admin' not in sess_data['roles']
# assert 'reader' in sess_data['roles']
```

Synthèse sprint 1

Une fois les 12 correctifs P0 appliqués, l'archive est **déployable sur Render en pré-production** (pas encore production défendable — les HIGH restants doivent être traités en sprints 2-3). L'estimation totale est de **3 à 5 jours-homme** répartis comme suit : P0-01 à P0-05 = 1 jour, P0-06 à P0-08 = 1 jour, P0-09 = 0.5 jour, P0-10 = 0.5 jour, P0-11 = 0.25 jour, P0-12 = 0.75 jour. Le sprint doit inclure une passe complète de tests (pytest + vitest + tsc + vite build) sur la branche `sprint-1-critical-fixes` avant fusion.

CHAPITRE 9 · PLAN DE REMÉDIATION P1

Plan de remédiation P1 — Haute priorité

Ce chapitre consolide les correctifs HIGH à traiter en sprints 2-3, après la branche P0. L'objectif est d'atteindre une posture de production défendable : migrations automatisées, observabilité, refactoring frontend minimal, tests E2E. L'effort total est estimé à **10 à 14 jours-homme** répartis sur 4 semaines.

P1-01 — Adopter Alembic pour les migrations

Fichiers : `migrations/alembic.ini`, `migrations/env.py`, `migrations/versions/0001_base_schema.py`.
Impact : remplace les `psql -f` manuels par `alembic upgrade head`. Ajoute une table `schema_migrations` qui track l'état. `preDeployCommand` dans render.yaml devient `alembic upgrade head`. Génère une migration de base depuis `Base.metadata` via `alembic revision --autogenerate` sur un PostgreSQL frais. Convertit les 3 migrations SQL existantes en revisions Alembic.
Effort : 2 jours.

P1-02 — Pin Actions à SHA et permissions least-privilege

Fichier : `.github/workflows/ci.yml`.
Impact : élimine le risque de supply-chain attack via Actions compromises. Pinner `actions/checkout`, `actions/setup-python`, `actions/setup-node`, `pnpm/action-setup`, `aquasecurity/trivy-action` à des SHA spécifiques. Ajouter `permissions: { contents: read }` sur les 4 jobs qui n'ont pas de block permissions.
Effort : 1 jour.

P1-03 — Sentry + structlog + rate limiting

Fichiers : `requirements.txt`, `web/app.py`, `.env.example`, `render.yaml`.
Impact : observabilité production. Sentry capture les exceptions non gérées. `structlog` remplace les `print()` par du JSON structuré. `slowapi` ajoute du rate limiting sur `/api/auth/dev-login` (5/min), `/api/auth/oidc/login` (10/min), `/api/auth/local/login` (5/min), `/api/auth/local/enroll` (5/hour).
Effort : 1.5 jour.

P1-04 — Content-Security-Policy header

Fichier : `web/app.py:174-183` (middleware `security_headers`).
Impact : ferme la surface XSS. Combiné avec P0-04 (suppression du token en sessionStorage) et P0-12 (CSP), l'attack surface XSS est drastiquement réduite. Voir le snippet de correctif au chapitre 5, section 5.12.

Effort : 0.5 jour (incluant audit des connect-src autorisés).

P1-05 — Remplacer tRPC shim par fetchTRPC typé

Fichiers : nouveau `frontend/client/src/lib/api.ts`, `frontend/client/src/lib/generated/openapi-types.ts` (généré via `openapi-typescript` depuis FastAPI OpenAPI schema), suppression de `frontend/server/routers.ts`, refactor de tous les `trpc.x.y.useQuery()` en hooks React Query typés.
Impact : restaure la safety de types. Renommage de procédure backend casse la compilation, pas le runtime.
Effort : 3 jours.

P1-06 — Lazy-load Home + extraction sous-vues

Fichiers : `frontend/client/src/App.tsx`, `frontend/client/src/pages/Home.tsx`.
Impact : divise le bundle initial de ~2 Mo à ~300-500 KB. TTI mobile < 3s au lieu de > 5s. Voir snippet chapitre 5, section 5.16.
Effort : 2 jours (incluant l'audit des imports dynamiques et la mise à jour des tests de snapshot).

P1-07 — Suite Playwright E2E (6 scénarios)

Fichiers : nouveau `frontend/e2e/playwright.config.ts`, `frontend/e2e/login.spec.ts`, `frontend/e2e/project_crud.spec.ts`, `frontend/e2e/save_reload.spec.ts`, `frontend/e2e/two_tab_conflict.spec.ts`, `frontend/e2e/oidc_callback_mock.spec.ts`, `frontend/e2e/labbench_step_up.spec.ts`.
Impact : couvre les flux utilisateur critiques. Élimine le statut BLOCKED sur R18 (browser E2E). Ajouter un job `e2e` dans la CI.
Effort : 3 jours.

P1-08 — Ajouter pgserver à requirements-dev.txt

Fichiers : nouveau `requirements-dev.txt`, `tests/test_v6_real_postgresql.py` (ajouter `pytest.importorskip('pgserver')`).
Impact : les 4 tests `test_v6_real_postgresql.py` ne ERROR plus en CI standard. Soit ils passent (si pgserver installé), soit ils SKIP proprement.
Effort : 0.5 jour.

P1-09 — Coverage threshold Vitest + raise maxfail

Fichiers : `frontend/vite.config.ts` (ajouter `test.coverage`), `.github/workflows/ci.yml:59` (retirer `--maxfail=5` ou monter à 20).
Effort : 0.25 jour.

P1-10 — Cleanup __manus__ leftovers + broken Tailwind class

Fichiers : suppression de `frontend/client/public/__manus__/`, suppression du host Manus dans `frontend/vite.config.ts:40`, fix de la classe Tailwind cassée dans `SuperAdmin.tsx:106` (`xl:grid-cols-inmax(0,1fr)_320px]` → `xl:grid-cols-[minmax(0,1fr)_320px]`).
Effort : 0.5 jour.

P1-11 — Document secret rotation runbook

Fichier : nouveau `docs/SECRET-ROTATION-FR.md`.
Contenu : cadence recommandée (90 jours OIDC, 180 jours session secret, 30 jours OpenAI), procédure step-by-step pour chaque secret (Render Dashboard → Environment → update → trigger redeploy → verify), procédure de rotation dual-key pour `AEROMIND_SESSION_SECRET` (déployer avec old+new acceptés pendant fenêtre de transition, puis retirer old).
Effort : 1 jour.

P1-12 — Écrire docs/PROCEDURE-INCIDENT-FR.md

Fichier : nouveau `docs/PROCEDURE-INCIDENT-FR.md` (référencé mais « à compléter » dans `PLAN-ROLLBACK-LABBENCH-01-FR.md:154`).
Contenu : rotation d'astreinte, classification de sévérité, templates de communication, modèle de postmortem.
Effort : 1 jour.

P1-13 — Mettre à jour le manuel opérateur

Fichier : `docs/MANUEL-OPERATEUR-FR.md` § 2.
Impact : corrige la contradiction C-11. Le manuel décrit un TOTP RFC-6238 par utilisateur renouvelé toutes les 30s, mais le code utilise un code quotidien émis par le super-admin via `verify_daily_totp_code`. Soit mettre à jour le manuel pour décrire le code quotidien, soit revert le code vers un vrai TOTP par utilisateur (plus sécurisé, aligné avec `local_auth.py`).
Effort : 0.5 jour.

Synthèse sprints 2-3

Une fois les 13 correctifs P1 appliqués (10-14 jours-homme), l'archive atteint un niveau de production défendable : migrations automatisées, observabilité, type safety frontend, tests E2E, secret rotation documenté. Le projet peut alors être déployé en pré-production Render (plan starter) pour validation finale.

CHAPITRE 10 · PLAN DE REMÉDIATION P2/P3

Plan de remédiation P2/P3 — Medium et Low

Ce chapitre consolide les 33 défauts MEDIUM et LOW à traiter en parallèle des sprints P0 et P1. Ils ne bloquent pas la mise en production mais alourdissent la maintenabilité, la robustesse ou la qualité perçue. La priorité est indicative — l'équipe peut réordonner en fonction du contexte. L'effort total est estimé à **8-12 jours-homme** répartis sur les sprints 4-6.

P2 — Défauts MEDIUM (priorité parallèle)

ID	Défaut	Fichier	Effort
P2-01	Ajouter HEALTHCHECK Dockerfile	Dockerfile	15 min
P2-02	Compléter .dockerignore	.dockerignore	15 min
P2-03	init: true + networks + mem_limit docker-compose	docker-compose.yml	30 min
P2-04	Log rotation json-file 10m×3	docker-compose.yml	15 min
P2-05	CSP preload HSTS	web/app.py:181	10 min
P2-06	Valider https:// origins au boot en prod	web/config.py	30 min
P2-07	Valider home_lat/home_lon avec Pydantic Field	web/app.py:1410-1428	30 min
P2-08	Cap polygon geofence à 200 points	web/app.py:1410-1428	30 min
P2-09	Cap maxAltitudeM à 120.0 (EASA Open)	web/geofence_validator.py:74-75	15 min
P2-10	Allowlist MAVLink router hosts (SSRF-lite)	web/mavlink_pool.py:155-171	1h
P2-11	Rate limit /api/local/workspace creation	web/app.py	1h
P2-12	Supprimer _issue_mission_item_token dead code	web/mission_planner.py:478-496	10 min
P2-13	Déclarer trpc_compat en def (threadpool)	web/app.py:473	15 min

ID	Défaut	Fichier	Effort
P2-14	Remplacer db.query legacy par db.scalars(select)	labbench_canonical_api.py + geofence_validator.py	2h
P2-15	Persist mechanical twin revisions to /api	MechanicalTwinEditor.tsx	4h
P2-16	Convert MutationObserver i18n à composants	LanguageContext.tsx	2 jours
P2-17	Re-key legacy.* i18n en namespaces	i18n/locales/*.json	2 jours
P2-18	Engager reviewer humain pour 183 fr===en	lib/fr.json + en.json	1 jour
P2-19	Remplacer window.confirm par	SuperAdmin.tsx, Home.tsx	1 jour
P2-20	Activer sourcemap: "hidden" en prod	vite.config.ts:47	15 min
P2-21	Supprimer sw.js dead code + fix manifest start_url	frontend/client/public/sw.js	15 min
P2-22	Remplacer OpenStreetMapView par maplibre-gl	OpenStreetMapView.tsx	3 jours
P2-23	Fix broken Tailwind class SuperAdmin.tsx:106	SuperAdmin.tsx:106	5 min
P2-24	Supprimer phantom dep "add" + "pnpm" devDep	frontend/package.json:103, 107	5 min
P2-25	Activer noImplicitAny: true dans tsconfig	tsconfig.json:11-12	1 jour (fix cast)
P2-26	Écrire docs/PROCEDURE-INCIDENT-FR.md (déjà en P1-12)	—	—
P2-27	Backup DB script + Render Cron Job	scripts/backup_db.sh	1 jour
P2-28	Corriger MANUEL-OPERATEUR MFA description (déjà en P1-13)	—	—
P2-29	Documenter AEROMIND_WORKSPACE_TTL_SECONDS etc.	.env.example	30 min
P2-30	Deduplicate i18n gate job (delete i18n-gate.yml)	.github/workflows/	15 min
P2-31	Add .nvmrc + frontend/.nvmrc	frontend/.nvmrc	5 min
P2-32	Re-enable CI blocking on HIGH/CRITICAL CVEs (déjà en P0-11)	—	—
P2-33	Restore validation/*.log OU stop citing them	validation/ + rapports	1 jour

Tableau 11 — Plan de remédiation P2 (33 défauts MEDIUM).

P3 — Défauts LOW (à traiter quand l'occasion se présente)

ID	Défaut	Fichier	Effort
P3-01	CMD shell form → exec form	Dockerfile:43	5 min
P3-02	Add .vscode/ .idea/ *.tsbuildinfo .DS_Store à .dockerignore	.dockerignore	5 min
P3-03	Remove app.config.ts (manus leftover)	app.config.ts	5 min
P3-04	Remove @builder.io/vite-plugin-jsx-loc unused	frontend/package.json:93	5 min
P3-05	Add copy_ctx=False aux canvas.drawString	web/app.py	15 min
P3-06	Replace print() by logging.getLogger	web/app.py + 8 fichiers	4h
P3-07	Move lazy imports to top-level	web/labbench_canonical_api.py	1h
P3-08	Add type hints to current_user	web/app_auth_helpers.py:7-24	15 min
P3-09	Persist _CERT_PRIVATE_KEY via env var	web/engineering_extensions.py:113	1h
P3-10	Add pytest.ini or pyproject.toml [tool.pytest]	—	30 min
P3-11	Add frontend/pnpm-workspace.yaml ou supprimer	frontend/pnpm-workspace.yaml	5 min
P3-12	Supprimer pages/StrataStudio.tsx + ComponentShowcase.tsx (dead)	frontend/client/src/pages/	15 min
P3-13	Replace catch {} silencieux par console.warn	frontend/client/src/ (40+ sites)	1 jour
P3-14	Add aria-busy + aria-live aux boutons et loading panels	LabBench.tsx + SuperAdmin.tsx	1 jour
P3-15	Remove console.error en prod (gate behind DEV)	main.tsx:35,43	15 min

Tableau 12 — Plan de remédiation P3 (15 défauts LOW).

CHAPITRE 11 · ROADMAP

Roadmap vers 100% opérationnalisation

La roadmap proposée s'étale sur 6 sprints de 2 semaines (12 semaines calendaires), avec un effort total estimé à 25-35 jours-homme. Elle est calibrée pour atteindre une mise en production défendable sur Render + PostgreSQL à la fin du sprint 6, conformément au périmètre opérationnel cible.

Vue d'ensemble — 6 sprints

Sprint	Période	Thème	Livrables clés	Effort
S1	Sem. 1-2	P0 — Critical fixes	12 correctifs bloquants (chap. 8)	3-5 j-h
S2	Sem. 3-4	P1 — DevOps hardening	Alembic, scanners bloquants, Actions SHA-pinnés, Sentry, CSP, slowapi	5-7 j-h
S3	Sem. 5-6	P1 — Frontend refactor	fetchTRPC typé, lazy-load Home, extraction sous-vues	5-7 j-h
S4	Sem. 7-8	P1 — Observability	structlog, Sentry hooks, /metrics, alerting	3-4 j-h
S5	Sem. 9-10	P1 — Tests	Playwright E2E (6 scénarios), coverage thresholds, pgserver fix	4-5 j-h
S6	Sem. 11-12	Pre-prod deploy	Render starter, OIDC real config, smoke tests, go-live	3-5 j-h

Tableau 13 — Roadmap 6 sprints vers 100% opérationnalisation.

Sprint 1 — P0 Critical Fixes (semaines 1-2)

Objectif : éliminer les 12 défauts CRITICAL qui bloquent la mise en production.
Livrables : branche `sprint-1-critical-fixes` avec les 12 correctifs du chapitre 8 appliqués. CI durcie (P0-11). Suite pytest + vitest verte. Tests de non-régression ajoutés pour P0-01 (current_user_or_401), P0-12 (step-up roles server-side).
KPIs : 0 défaut CRITICAL restant. Build Docker passe. `/api/lab/sessions/step-up` retourne 200 (pas 500) avec un user authentifié. `/api/hardware/pool/register` retourne 200 (pas 400).

Sprint 2 — P1 DevOps Hardening (semaines 3-4)

Objectif : infrastructure de production défendable.
Livrables : Alembic initialisé avec migration de base autogénérée. 3 migrations SQL existantes converties en revisions Alembic. `preDeployCommand: alembic upgrade head` dans render.yaml. CI scanners bloquants (P0-11 déjà fait). Actions SHA-pinnés. `permissions: { contents: read }` sur tous les jobs. Sentry intégré. `structlog` remplace les `print()`. `slowapi` sur endpoints auth. CSP header posé.
KPIs : 0 HIGH restant côté DevOps. `alembic upgrade head` passe sur un PostgreSQL frais.

> Sentry capture une exception de test.

Sprint 3 — P1 Frontend Refactor (semaines 5-6)

Objectif : restaurer la safety de types et réduire le bundle initial.
Livrables : `fetchTRPC` typé depuis OpenAPI (`openapi-typescript`). Suppression de `frontend/server/routers.ts` (AppRouter = any). Refactor des appels `trpc.x.y.useQuery()` en hooks typés. `App.tsx` lazy-load Home. Extraction de 5 sous-vues lourdes de `Home.tsx` en composants `lazy()` séparés.
KPIs : bundle initial < 500 KB (vs > 2 Mo actuellement). `pnpm run build` passe avec `noImplicitAny: true`. TTI mobile < 3s sur Lighthouse.

Sprint 4 — P1 Observability (semaines 7-8)

Objectif : visibilité production, alerting.
Livrables : `structlog` avec sortie JSON. Correlation ID middleware. Endpoint `/metrics` (`prometheus-fastapi-instrumentator`). Sentry hooks sur `app.exception_handler`. Render uptime monitor configuré avec alert email. Backup DB automatique Render starter + script `scripts/backup_db.sh` en Cron Job quotidien.
KPIs : une erreur 500 en pré-prod apparaît dans Sentry avec stack trace. `/metrics` expose latency p50, p95, p99.

Sprint 5 — P1 Tests E2E (semaines 9-10)

Objectif : couverture des flux utilisateur critiques.
Livrables : `playwright.config.ts` + 6 scénarios (login OIDC mock, project CRUD, save/reload, 2-tab conflict, LABBENCH step-up, measurement persistence). Job `e2e` dans CI (matrice chromium only). `requirements-dev.txt` avec `pgserver` + `pytest.importorskip`. Coverage thresholds vitest à `{ lines: 60, functions: 60, branches: 50 }`. `--maxfail=20` au lieu de 5.
KPIs : 6 scénarios E2E passent. 0 test ERROR en CI standard. Coverage frontend >= 60%.

Sprint 6 — Pre-production Deploy (semaines 11-12)

Objectif : go-live sur Render starter + PostgreSQL starter.
Livrables : render.yaml avec `plan: starter` pour web et DB. OIDC Microsoft Entra configuré avec un vrai tenant (variables `ENTRA_TENANT_ID`, `ENTRA_CLIENT_ID`, `ENTRA_CLIENT_SECRET`, `ENTRA_REDIRECT_URI` saisies via Render Dashboard). `AEROMIND_ALLOWED_HOSTS` et `AEROMIND_ALLOWED_ORIGINS` configurés avec le domaine Render exact. `AEROMIND_SUPERADMIN_EMAILS` saisi manuellement (pas de default). Smoke tests post-déploiement : `/api/ready` 200, `/api/health` 200, login OIDC callback OK, création de projet OK, session step-up OK.
KPIs : 0 CRITICAL, 0 HIGH non traité. 0 CVE HIGH/CRITICAL en CI. TTI < 3s. Coverage > 60%. Smoke tests post-déploiement verts. Audit trail `audit_events_ext` alimenté.

KPIs globaux de succès

À la fin du sprint 6, les KPIs suivants doivent être atteints :

KPI	Valeur actuelle	Cible sprint 6
Défauts CRITICAL	12	0
Défauts HIGH	31	0
Défauts MEDIUM	24	< 10 (P3 restant)
Défauts LOW	15	< 5
CVE HIGH/CRITICAL en CI	unknown	0 (bloquant)
Coverage frontend	~30% (lib only)	⩾ 60%
Coverage backend	~60%	⩾ 70%
Scénarios E2E Playwright	0	6 (tous verts)
Bundle initial frontend	~2+ Mo	< 500 KB
TTI mobile Lighthouse	> 5s	< 3s
Temps build CI	~8 min	< 5 min
Logs structurés JSON	0 (print)	100%
Sentry events capturés	non configuré	actif
Rate limiting auth	non configuré	5-10/min
Backup DB automatique	non configuré	quotidien 7j rétention
OIDC Entra configuré	non	oui (tenant réel)

Tableau 14 — KPIs globaux de succès (cible fin sprint 6).

CHAPITRE 12 · CONCLUSION

Conclusion et recommandations finales

Le projet AeroMind est architecturalement sain : 41 modules backend bien découpés, tests backend solides (30 fichiers pytest, ~210 tests), multi-stage Docker avec utilisateur non-root, RBAC à six rôles, gates G0-G5 définies, plan de rollback par lot documenté, lockfiles committés. La traçabilité des 75 rapports d'audit accumulés depuis le 13 octobre 2026 est exceptionnelle — mais elle révèle aussi que plusieurs défauts ont été identifiés à 4-8 reprises sans être corrigés, et que certaines déclarations de closure contredisent la réalité du code (SEC-05, validation/*.log, count tests).

Bilan global

L'inspection révèle **72 défauts actionnables** (12 CRITICAL, 31 HIGH, 24 MEDIUM, 15 LOW) répartis sur les quatre axes : backend (17 défauts), frontend (24), tests/audits (13), DevOps (18). La moitié des défauts CRITICAL sont des bugs d'intégration (fonction inexistante, argument manquant, secret mal lu) plutôt que des défauts d'architecture — l'effort de correction est faible (3-5 jours-homme pour le sprint 1) mais l'impact est immédiat. Les défauts HIGH concernent surtout l'observabilité (pas de Sentry, pas de /metrics, pas de rate limiting) et la dette frontend (Home.tsx 14k lignes, 0 test page, tRPC non typé) — correctifs plus longs mais indispensables pour une production défendable.

Cinq actions top-priority résumées

Si l'équipe ne pouvait appliquer que cinq correctifs, ce seraient les suivants — classés par impact et effort :

Action 1 — Débloquer le backend LABBENCH et le matériel (P0-01 + P0-02 + P0-03)

Trois correctifs simultanés : (a) définir `current_user_or_401` dans `web/app_auth_helpers.py` (4 lignes) — débloque 12 endpoints /api/lab/* ; (b) fixer `_hmac_secret` dans `web/labbench_auth.py` pour utiliser `get_settings().session_secret` — restaure l'intégrité des tokens matériels ; (c) passer `owner_id` et `project_id` dans `app.py:1202, 1227, 1304, 1427` — débloque pool register/unregister/command et geofence PUT. **Effort : 30 minutes.** Impact : débloque 90% des fonctionnalités LABBENCH et matériel.

Action 2 — Supprimer mcgregoralvin14@gmail.com partout (P0-10)

Cinq fichiers à modifier : `web/config.py:55,59,62`, `web/admin.py:65`, `.env.example:23-25`, `render.yaml:49-54`, `tests/test_demo_email_config.py:8-9`. Passer les valeurs par défaut à chaîne vide, et `sync: false` dans render.yaml pour saisie manuelle au premier déploiement. **Effort : 20 minutes.** Impact : élimine l'escalade de privilèges par défaut sur tout nouveau déploiement Render.

Action 3 — Set AUTO_CREATE_SCHEMA=false + preDeployCommand Alembic (P0-09 + P1-01)

Modifier `render.yaml:33-34` pour `AEROMIND_AUTO_CREATE_SCHEMA: "false"` et ajouter `preDeployCommand: "bash scripts/apply_migrations.sh $DATABASE_URL"` (ou `alembic upgrade head` après adoption). Créer le script `scripts/apply_migrations.sh`. **Effort : 30 minutes (P0) + 2 jours (Alembic complet en P1).** Impact : élimine le skew silencieux ORM/DB et rend les migrations reproductibles.

Action 4 — Make CI scanners blocking (P0-11)

Trois modifications dans `.github/workflows/ci.yml` : retirer `|| true` de `pip-audit` et `pnpm audit`, passer Trivy à `exit-code: "1"` et pinner `aquasecurity/trivy-action@0.20.0`. Ajouter `concurrency: { group: ci-${{ github.ref }}, cancel-in-progress: true }` et `timeout-minutes: 30` par job. Configurer branche protégée exigeant les 5 jobs CI verts. **Effort : 15 minutes.** Impact : une CVE critique RCE ne pourra plus être mergée sans révision humaine.

Action 5 — Add Sentry + CSP + rate limiting (P1-03 + P1-04)

Ajouter `sentry-sdk[fastapi]`, `structlog`, `slowapi` à `requirements.txt`. Initialiser Sentry dans le lifespan `web/app.py`. Ajouter `Content-Security-Policy` header dans le middleware `security_headers`. Configurer rate limiting sur `/api/auth/dev-login` (5/min), `/api/auth/oidc/login` (10/min). **Effort : 2 jours.** Impact : observabilité production (Sentry capture les 500 avec stack), fermeture de la surface XSS (CSP), protection brute-force (rate limiting).

Recommandations finales

1. Ne pas déployer en production tant que le sprint 1 n'est pas terminé. L'archive actuelle contient des bugs CRITICAL qui rendent plusieurs endpoints injoignables (current_user_or_401 manquant → tout le routeur /api/lab/* injoignable) et une vulnérabilité d'escalade de privilèges (SEC-08 step-up roles client-supplied). Tenter un déploiement Render maintenant c'est accepter un service cassé en façade et une attaque potentielle en dessous.

2. Adopter une discipline de closure stricte. Plusieurs défauts (mcgregoralvin14@gmail.com, Home.tsx monolithe, validation/*.log vide, count tests 273 vs 281 vs 339) ont été déclarés corrigés dans un rapport puis identifiés à nouveau comme non corrigés dans le suivant. Cette instabilité nuit à la confiance dans les déclarations de closure futures. Recommandation : tout rapport de closure doit inclure un test automatisé qui vérifie le correctif, et ce test doit être exécuté en CI avant fusion.

3. Traiter la dette frontend dès le sprint 3. Home.tsx (14 315 lignes) est le plus gros fichier du projet et n'a aucun test. Il a grandi de 10% depuis l'audit du 14 octobre qui recommandait son éclatement. Plus le fichier grossit, plus le refactor sera coûteux. Le lazy loading et l'extraction des sous-vues sont des prérequis pour tout travail futur sur le cockpit.

4. Investir dans les tests E2E navigateur dès le sprint 5. L'absence de suite Playwright est un défaut récurrent depuis le 13 octobre, identifié à 8 reprises comme BLOCKED sans résolution. C'est le seul moyen de couvrir les flux OIDC callback, 2-tab conflict, F5 restore et step-up MFA — qui sont les flux les plus risqués pour la sécurité et l'UX. Le coût (3 jours-homme) est faible par rapport au risque d'une régression silencieuse sur un de ces flux.

5. Planifier un audit externe après le sprint 6. Une fois les 5 actions top-priority et les sprints 2-6 terminés, l'archive atteint un niveau de production dépassant la majorité des projets internes. Un audit externe indépendant (par une équipe n'ayant pas écrit le code) permettrait de valider la posture de sécurité et de détecter les défauts résiduels avant une mise en production à plus grande échelle. C'est aussi un prérequis si AeroMind doit un jour être soumis à une conformité réglementaire (EASA, FAA, SOC 2).

Mot de la fin

Le projet AeroMind a atteint un niveau de maturité architecturale et de traçabilité remarquable pour son ampleur. Les 72 défauts identifiés par la présente inspection ne remettent pas en cause l'architecture — ils sont pour la majorité des bugs d'intégration ou des oublis de configuration, rapidement corrigeables. Le plan de remédiation proposé (6 sprints, 25-35 jours-homme) permet d'atteindre une mise en production défendable sur Render + PostgreSQL dans un délai de 12 semaines. Au-delà des corrections techniques, le projet gagnerait à adopter une discipline de closure stricte : tout défaut déclaré corrigé doit être vérifié par un test automatisé, et tout rapport d'audit doit citer ses preuves (fichiers, lignes, logs) de manière vérifiable. Cette discipline est la clé pour transformer l'excellente traçabilité accumulée (75 rapports) en confiance opérationnelle réelle.